

南京工业大学

2026 届毕业设计（论文）

题 目：面向 F2FS 文件系统的

Large Folios 机制研究与实现

专 业：人工智能

班 级：智能 2202

姓 名：赵南哲

指导老师：万夕里

起讫日期：2026.3 至 2026.6

2026 年 5 月

南京工业大学本科毕业设计（论文）原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

论文作者签名： 日期： 年 月 日

南京工业大学本科毕业设计（论文）使用授权声明

本学位论文作者完全了解学校有关保留、使用毕业论文（设计）的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权南京工业大学教务处可以将本毕业论文（设计）的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编毕业论文（设计）。

论文作者签名： 导师签名：
日期： 年 月 日 日期： 年 月 日

面向 F2FS 文件系统的 Large Folios 机制研究与实现

摘 要

在现代移动设备上，应用启动、资源加载和端侧模型加载等场景对文件系统连续 I/O 能力提出了更高要求。F2FS（Flash-Friendly File System，闪存友好文件系统）作为安卓设备中常用文件系统，其缓冲 I/O 路径仍以 4KB 页和文件系统块为基本处理单位，难以充分发挥底层存储设备的大粒度顺序 I/O 能力，从而限制了大文件连续访问场景下的吞吐提升。为此，F2FS 需要在块级映射、页缓存状态和垃圾回收、压缩文件等专属路径之间建立面向动态大页的适配机制。

为解决上述问题，本文将动态大页引入 F2FS 缓冲 I/O 路径，并设计实现一套基于 iomap 的动态大页缓冲 I/O 适配框架。该框架将 F2FS 块级映射结果转换为文件字节区间的 iomap 描述，扩展动态大页内部状态对象，以子范围维护脏状态、就绪和 I/O 完成状态；同时适配垃圾回收和压缩文件路径，使后台块搬运和簇级压缩处理能够与普通缓冲 I/O 共享一致的动态大页语义。

本文在 QEMU 虚拟机和树莓派平台上对原型系统进行了实验评估。结果表明，相比未接入 iomap 动态大页路径的基线系统，本文框架在大块顺序 I/O 场景下能够显著提升 F2FS 的吞吐能力：QEMU 平台普通文件写入带宽提升约 2-3 倍，树莓派平台普通文件、稀疏文件和压缩文件写入场景最高提升约 3-4 倍。同时，CPU 开销和带宽稳定性结果表明，该框架能够降低小粒度页缓存管理和逐块 I/O 组织带来的额外开销，验证了 F2FS 支持动态大页缓冲 I/O 的可行性和有效性。

关键词：F2FS 动态大页 页缓存 iomap 缓冲 I/O

Research and Implementation of the Large Folios Mechanism for the F2FS File System

ABSTRACT

On modern mobile devices, application startup, resource loading, and on-device model loading place higher demands on file-system sequential I/O. F2FS, a flash-friendly file system widely used on Android devices, still organizes its buffered I/O path around 4 KB pages and file-system blocks, which limits its ability to exploit large-granularity sequential I/O capability of underlying storage devices. Therefore, F2FS requires an adaptation mechanism for large folios across block mapping, page-cache state management, garbage collection, compressed files, and other file-system-specific paths.

To address this issue, this thesis introduces large folios into the F2FS buffered I/O path and designs an iomap-based large-folio buffered I/O adaptation framework. The framework converts F2FS block-level mapping results into iomap descriptions over file byte ranges, extends the internal state object of large folios, and maintains dirty, uptodate, and I/O completion states at subrange granularity. It also adapts garbage collection and compressed-file paths so that background block migration and cluster-level compression processing share consistent large-folio semantics with ordinary buffered I/O.

The prototype is evaluated on QEMU virtual machines and a Raspberry Pi platform. Results show that, compared with the baseline system without the iomap-based large-folio path, the proposed framework significantly improves F2FS throughput under large-block sequential I/O workloads: ordinary-file write bandwidth on QEMU improves by about 2-3 times, while ordinary-file, sparse-file, and compressed-file write scenarios on Raspberry Pi improve by up to 3-4 times. CPU-overhead and bandwidth-stability results further show that the framework reduces the extra overhead caused by fine-grained page-cache management and block-by-block I/O organization, validating the feasibility and effectiveness of supporting large-folio buffered I/O in F2FS.

Key Words: F2FS; Large Folio; Page Cache; iomap; Buffered I/O

目 录

摘要	I
ABSTRACT	II
第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状与挑战	2
1.2.1 页粒度与大页机制相关研究	2
1.2.2 文件系统映射与 iomap 相关研究	3
1.2.3 F2FS 与闪存友好文件系统相关研究	4
1.3 大块 I/O 与更大内存单元需求	5
1.4 本文主要工作	5
1.5 论文组织结构	6
第二章 背景知识与关键技术	7
2.1 Linux 缓冲 I/O 路径	7
2.1.1 页缓存与文件偏移管理	7
2.1.2 F2FS 块映射过程	7
2.1.3 块设备 I/O 请求 (bio) 提交与异步完成钩子	9
2.1.4 64KB 缓冲读路径示例	9
2.2 F2FS 的闪存友好机制与专属路径	10
2.2.1 闪存写擦除约束与异地更新	10
2.2.2 垃圾回收机制	12
2.2.3 压缩文件机制	12
2.3 Large Folio (动态大页) 机制	12
2.4 iomap 区间映射与动态大页状态管理	14
2.4.1 iomap 区间映射模型	14
2.4.2 iomap 框架及迭代机制	15
2.4.3 iomap 动态大页状态对象与子范围状态管理	16
2.5 本章小结	18
第三章 F2FS 动态大页缓冲 I/O 方案设计	19

3.1 设计目标与总体思路·····	19
3.2 系统总体架构·····	19
3.3 普通文件区间化映射机制设计·····	20
3.4 扩展型动态大页状态对象设计·····	21
3.5 子范围脏区与异步完成管理设计·····	23
3.6 压缩文件路径适配策略·····	24
3.7 GC 路径适配策略·····	26
3.8 本章小结·····	27
第四章 F2FS 动态大页缓冲 I/O 实现·····	28
4.1 普通文件区间化读写实现·····	28
4.1.1 块映射到 iomap 区间的转换实现·····	28
4.1.2 普通文件缓冲读钩子实现 ·····	29
4.1.3 普通文件缓冲写钩子实现 ·····	31
4.2 扩展型动态大页状态对象实现 ·····	32
4.3 压缩文件动态大页读写实现 ·····	34
4.3.1 压缩文件缓冲读实现 ·····	35
4.3.2 压缩文件缓冲写实现 ·····	38
4.4 动态大页子范围写回实现 ·····	40
4.4.1 普通文件子范围写回实现 ·····	40
4.4.2 压缩文件写回与延迟解锁实现 ·····	42
4.5 GC 路径动态大页适配实现·····	43
4.6 本章小结 ·····	44
第五章 实验设计与性能评估·····	45
5.1 测试环境与配置 ·····	45
5.2 测试方法与负载设计 ·····	47
5.2.1 工作负载 ·····	47
5.2.2 测试矩阵 ·····	47
5.2.3 评价指标 ·····	48
5.3 实验结果与分析 ·····	48
5.3.1 QEMU 虚拟机平台·····	48

5.3.2 树莓派 5 平台	52
5.3.3 CPU 利用率分析	62
第六章 总结与展望	65
6.1 工作总结	65
6.2 未来展望	66
参考文献	66
致谢	69

第一章 绪论

1.1 研究背景与意义

近年来，移动终端本地数据规模持续增长，端侧模型加载、影像处理、媒体资源管理等场景逐渐增加了连续大文件访问需求，因此对文件系统的要求越来越高。这些应用场景往往涉及较长文件区间的顺序读取、缓存填充和批量写回，对文件系统的顺序 I/O 能力、页缓存管理效率以及路径开销控制提出了更高要求^[1]。

F2FS（Flash-Friendly File System，闪存友好文件系统）是安卓设备中目前使用的主流文件系统，它通过日志结构写入、冷热数据分离和垃圾回收等机制适应闪存设备。实际使用中，F2FS 不仅负责普通应用数据读写，也会参与模型文件、媒体文件和资源文件的加载过程。随着移动设备存储容量增大，单次访问的数据范围也比过去更大，因此需要对其进行优化^[2]。

F2FS 现有的缓冲 I/O 仍然主要围绕 4KB 页和 4KB 文件系统块组织。一次 64KB 的连续访问，在页缓存层可能对应 16 个页缓存对象，在文件系统层也要处理 16 个逻辑块。即使块设备层最后能够合并部分请求，页缓存层和文件系统层仍然要经历多次查找、加锁、状态更新和完成回调。对于小文件或随机访问，这种方式比较直接；但对于大块顺序访问，重复的页缓存和块映射处理就会变成额外开销。

Linux 的动态大页（Large Folio）机制为页缓存使用更大的缓存对象提供了条件。一个大页（高阶 folio）可以覆盖多个连续 4KB 页，从而减少页缓存对象数量。但这只是页缓存层面的变化，并不意味着文件系统内部可以自动按大粒度工作。以 64KB folio 为例，它在页缓存中是一个对象，但在 F2FS 中仍然对应 16 个 4KB 文件系统块。文件系统仍然要知道这些子块分别映射到哪里，哪些范围已经读入，哪些范围被修改，以及哪些 I/O 已经完成。

除了普通文件读写之外，F2FS 还有压缩文件和垃圾回收（GC）路径。压缩文件按簇组织数据，读取时需要先读压缩块再解压填充页缓存；GC 中的垃圾搬移则以有效块为单位重新读取和写入数据。如果页缓存对象变大，而这些路径仍然按原来的单页或单块假设处理，就可能出现子范围状态不清楚、完成

回调提前结束、GC 搬移位置计算错误等问题。因此，F2FS 接入动态大页时，需要同时处理普通读写、写回、压缩文件和 GC 搬移之间的状态一致性。

基于上述背景，本文尝试在 F2FS 文件系统中接入动态大页，并使用 iomap 框架描述更大的文件区间映射。本文并不改变 F2FS 的磁盘布局，而是在原有 4KB 块语义基础上，让一个动态大页内部的多个子块能够分别记录映射、脏状态和 I/O 完成情况，使页缓存层的大粒度缓存能力能够与 F2FS 文件系统内部机制相衔接，并且实现普通文件读写、子范围写回、压缩文件读写和 GC 搬移几个关键技术。

1.2 国内外研究现状与挑战

1.2.1 页粒度与大页机制相关研究

大基页机制通过直接扩大操作系统基础页粒度，减少同等虚拟地址空间下的页表项数量和快表缺失次数，从而降低地址转换和页表遍历开销。Ganapathy 等人较早研究了通用操作系统对多种页大小的支持，指出多页大小能够提升快表覆盖范围并改善大工作集程序的执行效率，但其系统支持还涉及分配、映射和兼容性等多方面约束^[3]。Winwood 等进一步讨论了 Linux 内核中的多页大小支持，说明该类机制需要在虚拟内存子系统中引入页大小选择、页表构造和缺页处理等配套修改^[4]。后续关于 4KB 基页局限性的研究也表明，在内存容量和工作集规模持续增长的背景下，适当增大基础页大小有助于扩大快表覆盖范围，但也需要在内部碎片、内存占用和 I/O 传输粒度之间权衡^[5]。在 Android 生态中，AOSP 已支持 16KB 基页大小，该配置能够改善部分系统性能，但会增加内存占用，并要求用户态组件以 16KB ELF 对齐方式构建^[6]。对于依赖 NDK 或第三方 .so 的应用，若本地库仍按照 4KB 页假设编译、对齐或在代码中硬编码页大小，则需要重新构建或修改相关假设，否则可能无法在 16KB 页设备上可靠运行^[7]。因此，大基页机制虽然能够从系统基准粒度层面降低页表管理和地址转换开销，但它以改变全局内存管理粒度为前提，对既有二进制接口、用户态库和应用生态具有较强侵入性，难以作为面向特定文件访问路径的局部优化手段。

透明大页（transparent hugepage）机制则试图在不要求应用显式申请大页的情况下，由内核在运行时自动将连续虚拟地址范围映射为更大的物理页，从而在保持应用接口透明的同时获得部分大页收益。Navarro 等提出的透明

superpage 支持说明，操作系统可以在应用无感知的条件下利用大页降低快表压力^[8]。围绕透明大页的后续研究也表明，大页的自动提升、拆分、回收和放置策略会显著影响实际性能；若缺乏细粒度管理，透明大页可能因错误提升、回收困难和局部性不匹配而带来不稳定收益，甚至在部分负载下引入额外开销^{[9][10]}。与直接启用大基页相比，透明大页的主要局限在于其依赖运行时连续物理内存分配和压缩整理，碎片化问题会影响大页形成、维持和回收过程，这是透明大页机制自身较突出的缺点。进一步看，无论是大基页还是透明大页，其分配粒度通常围绕体系结构或内核策略给定的固定大小展开，优化目标主要集中在虚拟内存层面的地址转换和页表管理，而不是根据文件访问模式、预读窗口、页缓存命中状态和写回子范围进行自适应调整。因此，这两类机制虽然体现了扩大内存管理粒度的基本思路，但不能直接解决大型文件页缓存路径中缓存对象大小、文件系统块映射和 I/O 提交粒度之间的协同问题。

1.2.2 文件系统映射与 iomap 相关研究

历史上用于描述页缓存与底层设备块之间的关系的数据结构是缓冲描述符（buffer_head）。一个页缓存会对应四个组织在一起的缓存描述符。在缓存描述符出现的时代，大部分存储设备还是以固定字节大小为 512 字节为一个扇区的固态硬盘为主，缓冲描述符非常直观地表示了一个扇区的软件抽象。但一方面随着能进行大块顺序 I/O 的固态硬盘的流行以及页缓存对象和 I/O 请求粒度增大，缓冲描述符这种使一个页缓存映射到多个块数据结构的模型设计会增加状态维护和路径组织成本。

为了改善大块 I/O 的性能，iomap 框架提出了区间（extent）映射思想^[11]。其直接以文件逻辑字节区间为单位描述底层映射关系，并通过统一的迭代器与钩子接口同时支持缓冲 I/O、直接 I/O（direct I/O）和脏页写回（脏状态 page writeback）等路径。XFS 是一种面向高性能和可扩展性的日志型文件系统，较早全面采用 iomap；ext4 和 ext2 分别是 Linux 中常用的第四代、第二代扩展文件系统，也逐步围绕直接 I/O 或缓冲 I/O 迁移到 iomap 框架^[12]。与此同时，Btrfs（B 树文件系统）等现代 Linux 文件系统也体现出以区间映射和写时复制为核心的文件系统组织趋势^[13]。已有实践表明，区间化映射方式能够减少对每

块状态对象的依赖，更适合与动态大页、大块顺序 I/O 和大块设备访问能力相结合^[14]。

现有研究已经说明了 iomap 的抽象优势和在主流文件系统中的推广趋势，但面向 F2FS 的协同适配仍处于持续演进阶段。具体而言，如何在 F2FS 的块级映射与 iomap 字节区间语义之间建立稳定的转换路径，如何在 iomap 逐动态大页维护的状态框架中容纳 F2FS 的 GC 和原子写等私有语义，以及如何使写回和压缩路径的完成判断适配 iomap 的原子计数机制，这三个层面的工程适配仍缺乏系统化的实现方案。

1.2.3 F2FS 与闪存友好文件系统相关研究

F2FS 最初作为面向 NAND 闪存设计的日志结构文件系统提出，其设计目标是在块设备抽象之上尽量保持顺序追加写入，降低随机覆盖更新带来的写放大，并借助冷热数据分离、多日志和垃圾回收机制改善闪存存储效率^{[15][16]}。相关研究和官方文档对 F2FS 的节点地址表（Node Address Table, NAT）、段信息表（Segment Information Table, SIT）、段摘要区（Segment Summary Area, SSA）、检查点（Checkpoint）以及段（Segment）、节（Section）、区（Zone）等空间组织方式进行了较系统说明，也讨论了其在移动设备和通用闪存场景中的应用价值^{[15][17]}。

除基础布局外，已有研究还关注了 F2FS 垃圾回收、压缩文件、冷热数据分离和块分配策略等问题。这些机制提升了 F2FS 对闪存设备的适应能力，但也使其文件访问路径较普通文件系统更复杂。例如，异地更新意味着写回并非原地覆盖，垃圾回收需要读取并搬移有效块，压缩文件则引入了簇级处理语义^[18]。这些特性决定了 F2FS 在面对更大粒度页缓存对象时，不能简单沿用传统单页或单块路径。

总体而言，已有工作已经分别说明了大页机制、iomap 区间映射框架和 F2FS 闪存优化设计的必要性：现有研究证明了 4KB 粒度在连续访问场景中的局限，也为动态大页提供了基础；iomap 框架展示了区间化 I/O 组织的通用优势；但针对 F2FS 场景下如何将动态大页、iomap 与 F2FS 自身的 GC、压缩和私有状态机制协同适配，尚缺乏面向完整关键路径的设计与实现。

1.3 大块 I/O 与更大内存单元需求

除虚拟内存层面的大页机制外，存储系统中的大块 I/O 需求也促使文件系统重新审视页缓存对象与文件系统块之间的粒度关系。KernelNewbies 的大型块大小（Large block size, 简称 LBS）资料将该问题概括为块大小大于页大小（bs > ps）场景：随着存储介质和设备接口演进，文件系统块、设备逻辑块以及应用期望的顺序 I/O 粒度都可能超过传统 4KB 页大小，因而需要内核在页缓存和文件系统路径中支持更大的数据承载单元^[19]。相关讨论还指出，Linux 过去长期将文件系统最大块大小限制在宿主页大小以内；后续大型块大小补丁在 XFS 中解除该限制，使文件系统块大小能够与系统页大小解耦，并以大粒度 folio 作为页缓存中承载单个大块的基本对象^[20]。

这一方向表明，提升连续文件访问性能不能仅依赖块层合并请求。若文件系统块或设备可寻址单元大于页大小，页缓存需要保证用于表示同一文件系统块的内存范围不会被拆分或局部回收，写回路径也需要以完整块或可验证的子范围为单位维护一致性。对于本文关注的 F2FS 缓冲 I/O 路径而言，虽然目标并非直接改变磁盘块大小，但大块 I/O 与 LBS 研究说明了更大内存单元在页缓存、文件系统块映射和 I/O 提交之间的桥接作用：只有当页缓存对象能够覆盖更长连续文件区间，并且文件系统能够精确维护其内部子范围状态时，底层存储的大粒度顺序访问能力才能有效传递到缓冲 I/O 路径。

1.4 本文主要工作

围绕上述研究背景与挑战，本文面向 F2FS 缓冲 I/O 路径设计并实现了一套动态大页适配方案。该方案首先将 F2FS 原有块级映射结果转换为 iomap 所需的文件字节区间，使普通文件读写能够围绕动态大页覆盖范围组织映射查询、空间预留和 I/O 提交；同时在页缓存对象内部扩展状态承载方式，将 F2FS 私有标志、子范围就绪/脏状态位图和读写完成计数统一到动态大页状态对象中，以支撑大粒度页缓存条件下的状态判断和写回控制。

在此基础上，本文进一步将该适配方案扩展到压缩文件和垃圾回收路径，使压缩簇处理、有效块搬移和普通缓冲 I/O 共享一致的子范围状态语义。最后，本文在 QEMU 虚拟机和树莓派平台上进行实验验证，对比改造前后 F2FS 在顺序

读写场景下的吞吐能力、带宽稳定性和 CPU 开销，评估动态大页缓冲 I/O 方案的实际效果。

1.5 论文组织结构

第一章为绪论，主要介绍本文的研究背景、国内外研究现状与挑战、大块 I/O 与更大内存单元需求，并概述本文的主要工作和全文结构。

第二章为背景知识与关键技术，主要介绍 Linux 缓冲 I/O 路径、F2FS 闪存友好机制、动态大页和 iomap，为后续动态大页缓冲 I/O 方案提供基础支撑。

第三章为 F2FS 动态大页缓冲 I/O 方案设计，围绕设计目标、总体架构、区间化映射、统一状态对象、子范围写回以及压缩文件与 GC 路径适配等方面，说明本文方案的机制组成。

第四章为 F2FS 动态大页缓冲 I/O 关键路径实现，重点说明普通文件读写钩子、扩展型状态对象分配与迁移、压缩文件读写、动态大页子范围写回和 GC 搬移适配的具体实现。

第五章给出实验设计与性能评估结果，说明测试环境、工作负载、评价指标与结果分析，并验证本文方案在顺序 I/O 场景下的有效性。

第六章为总结与展望，对全文工作进行总结，并围绕动态大页阶数选择、压缩路径优化、GC 协同和更多平台验证等方向提出后续展望。

第二章 背景知识与关键技术

2.1 Linux 缓冲 I/O 路径

2.1.1 页缓存与文件偏移管理

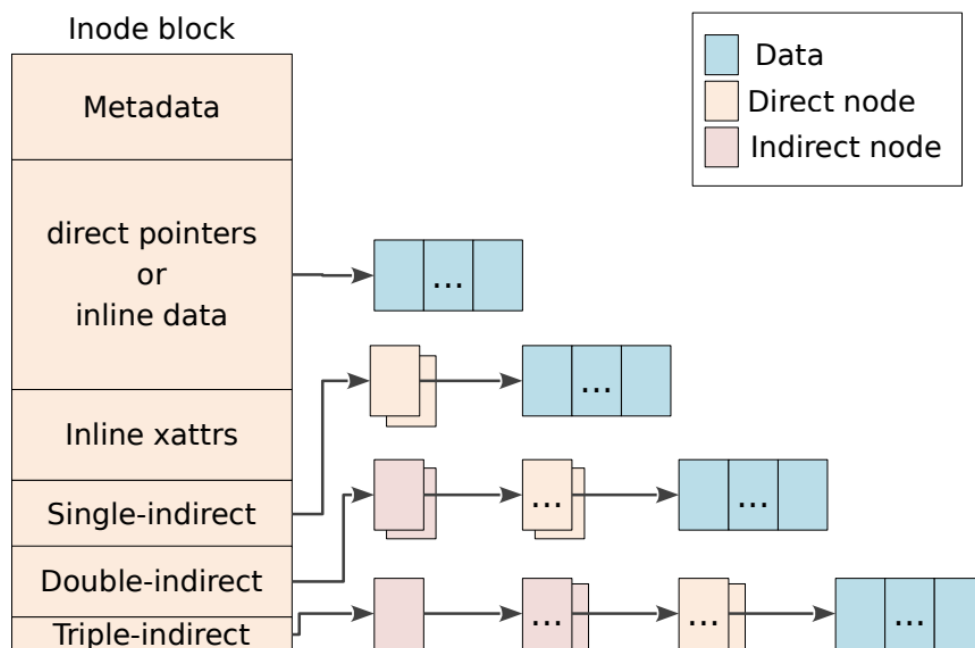
在 Linux 内核中，普通文件的缓冲读写首先经过页缓存^[21]。页缓存位于虚拟文件系统层和具体文件系统之间，用内存中的缓存对象保存文件数据，减少对底层存储设备的直接访问。应用发起 `read()` 或 `write()` 系统调用时，内核会根据文件偏移在文件的地址空间中查找对应缓存对象；若目标数据已经在页缓存中并处于就绪状态，则可以直接完成用户缓冲区与页缓存之间的数据拷贝；若页缓存未命中，则需要由文件系统提供磁盘映射，并通过块设备层发起实际 I/O。

传统页缓存路径长期以 4KB 页描述符（`struct page`）为基本管理单位。页描述符中的 `index` 表示该页在文件中的页索引，`mapping` 指向所属文件的地址空间，`flags` 则记录对象生命周期状态，例如 `locked`、`uptodate`、`dirty` 和 `writeback`^[22]，即锁定、就绪、脏页、写回状态标志位。锁定标志位表示对象的页缓存锁已经被持有，就绪标志位表示内存数据已从磁盘被读取，脏页标志位表示对象内容被修改但尚未写回，写回标志位表示该对象正处于写回落盘过程中。

在 4KB 页缓存粒度下，页缓存查找、分配、加锁、状态更新和映射维护都围绕单个页描述符展开。对于连续访问负载而言，上层管理粒度决定了文件系统映射调用、页状态维护以及 I/O 完成处理需要重复执行的次数。

2.1.2 F2FS 块映射过程

页缓存能够确定本次访问对应的文件偏移范围，但并不知道该范围在存储设备上的物理位置。这个映射关系由文件系统层提供。F2FS 使用基于节点的索引结构维护文件逻辑块到物理块的映射关系，每个文件的索引节点保存直接数据块指针和间接节点指针，间接节点继续指向更下一级节点或数据块，由此形成多级索引树。

图 2-1 F2FS 索引节点与数据块映射结构(来源^[15])

当页缓存请求文件系统为某个文件偏移范围提供映射时，F2FS 的块映射入口函数 `f2fs_map_blocks` 首先将文件偏移转换为文件内部逻辑块号，然后从索引节点出发沿节点树逐级查找，直至定位到目标逻辑块对应的直接节点条目。该条目中的块地址字段决定了该逻辑块当前对应有效物理块、空洞、新分配块还是压缩簇。

F2FS 块映射结果不仅包含物理块号，还包含该逻辑块的状态语义。有效物理块地址表示数据已经落盘并具有确定物理位置；`NULL_ADDR` 表示文件空洞或尚未分配的逻辑位置；`NEW_ADDR` 表示逻辑块已预留或新分配但数据尚未写入磁盘；`COMPRESS_ADDR` 表示该逻辑块属于压缩簇，需要按照压缩文件路径解析。上述状态与 `iomap` 区间类型之间的关系如表 2-1 所示。

表 2-1 F2FS 块映射状态与 iomap 区间类型对应关系

F2FS 块映射状态	含义	对应 iomap 区间类型或处理路径
有效物理块地址	数据已落盘，具有确定物理位置	IOMAP_MAPPED
NULL_ADDR	无物理映射，表示文件空洞或尚未分配的位置	IOMAP_HOLE
NEW_ADDR	已预留或新分配，但数据尚未真正写入磁盘	IOMAP_UNWRITTEN
COMPRESS_ADDR	逻辑块属于压缩簇，不能按普通物理块解释	压缩文件路径单独处理

2.1.3 块设备 I/O 请求（bio）提交与异步完成钩子

文件系统获得块映射后，内核需要向块设备层提交实际 I/O。Linux 块设备层使用块设备 I/O 请求（bio）描述一次块设备访问^[23]。该请求的 bi_iter 记录目标设备起始扇区和请求长度，bio_vec 数组描述本次 I/O 覆盖的内存区间。一个块设备 I/O 请求可以包含多个 bio_vec；在支持多页扩展 bvec 的实现中，单个 bio_vec 也可以覆盖一个多页内存区间。

因此，块设备层本身具备聚合多页 I/O 的能力。对于物理连续的 64KB 内存区间，块设备 I/O 请求可以使用较少的 bio_vec 描述并一次提交给块设备队列。块设备驱动完成 DMA 传输后，通过预先注册的完成钩子异步通知上层。读完成钩子通常负责设置页缓存对象的就绪状态并解锁对象，写完成钩子则清除写回状态位，使该对象退出写回生命周期。

异步完成机制意味着提交 I/O 的路径和完成 I/O 的路径并不在同一控制流中执行。完成钩子只能依据块设备 I/O 请求覆盖范围更新相应内存对象的状态，而页缓存对象内部是否还存在未完成的其他子范围，需要由更高层状态结构进行记录。

2.1.4 64KB 缓冲读路径示例

以一次从文件偏移 0 开始读取 64KB 数据的用户态请求为例，页缓存首先根据文件偏移计算页索引并查找缓存对象。若采用传统 4KB 页缓存粒度，64KB 范围对应 16 个页描述符；预读路径可能一次性分配 index=0 到 index=15 的页描述符，并分别对这些页加锁。

随后，页缓存调用 F2FS 预读路径获取逻辑块到物理块的映射。F2FS 将文件偏移转换为逻辑块号，从索引树中查找对应直接节点条目，并在物理块连续时返回覆盖多个逻辑块的映射结果。块设备层根据映射结果构建块设备 I/O 请求，将目标物理块号换算为设备扇区号，并把多个页对应的内存区间加入请求。I/O 完成后，钩子遍历请求覆盖的页缓存对象，更新其就绪状态并解锁，最后由系统调用把页缓存中的数据返回给用户缓冲区。图 2-2 展示了这一链路。

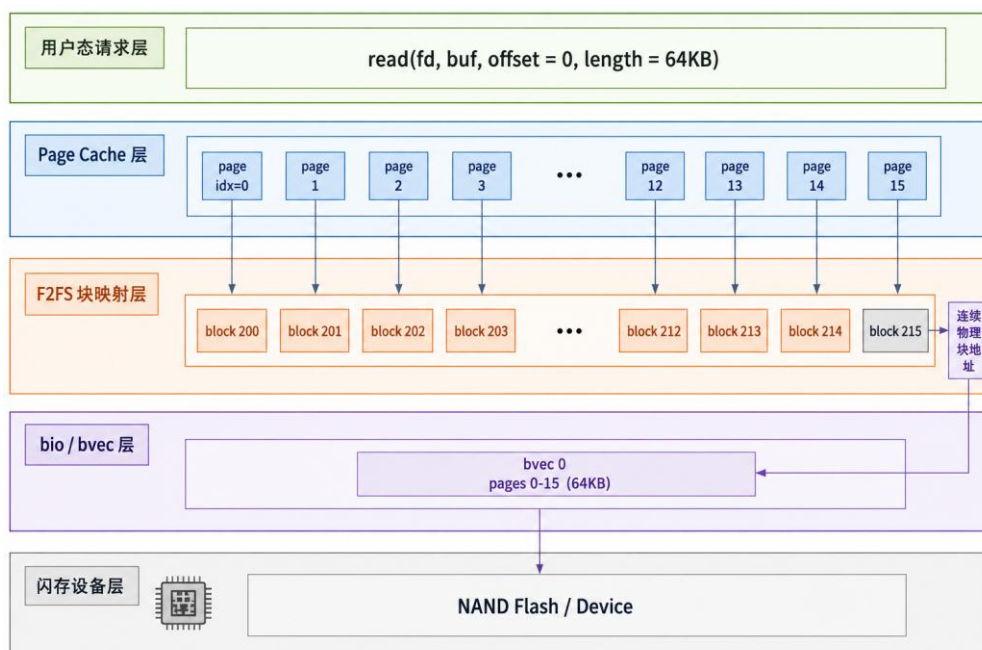


图 2-2 64KB 缓冲读完整路径分层图

该示例说明，块设备层可以把连续页组织为较大的块设备 I/O 请求，但在传统 4KB 页缓存粒度下，页缓存对象管理和文件系统映射维护仍以单页为基本单位执行。动态大页和 iomap 分别从页缓存对象粒度和区间化映射表达两个方面为后续适配提供基础。

2.2 F2FS 的闪存友好机制与专属路径

2.2.1 闪存写擦除约束与异地更新

F2FS 的闪存友好特性建立在 NAND 闪存的写入和擦除规则之上，因此需要先介绍闪存的基本写入特性，再说明 F2FS 如何围绕这些特性在文件系统层面组织数据更新。NAND 闪存写入数据时以页为单位，但页中的旧数据不能被新数据覆盖。同一物理页如果需要重新写入，必须先擦除其所在的擦除块，

而一个擦除块通常包含多个页。因此上层一次随机覆盖写，需将整个物理块擦除之后写入。

若文件系统频繁把小块数据写回到原物理位置，闪存转换层（Flash Translation Layer, FTL）需要先找出该擦除块中仍然有效的数据，将这些数据搬移到新的空闲位置，再进行旧块擦除和回收。这个过程会带来额外的数据搬运和擦除操作，形成写放大，也会加快闪存块磨损。

F2FS 因此采用异地更新方式。文件内容发生修改时，新数据写入当前活跃段中的空闲位置；写入完成后，该逻辑块的映射关系改指向新的物理块，旧物理块则在文件系统元数据中被标记为无效。

为了维护这种更新方式，F2FS 使用多类元数据共同记录最新数据物理块地址以及有效/无效物理块信息，并为数据搬移提供决策基础。节点地址表（Node Address Table, NAT）记录节点块的当前位置，使目录项和索引节点更新后仍能找到最新版本；段信息表（Segment Information Table, SIT）记录段内块的有效性，用于判断一个段中还有多少数据需要保留；段摘要区保存段内物理块与上层对象之间的反向信息，便于垃圾回收时根据物理块追溯对应的文件和逻辑位置；检查点机制则保存一组可恢复的一致元数据状态。图 2-3 展示了这些区域在 F2FS 磁盘布局中的位置关系：

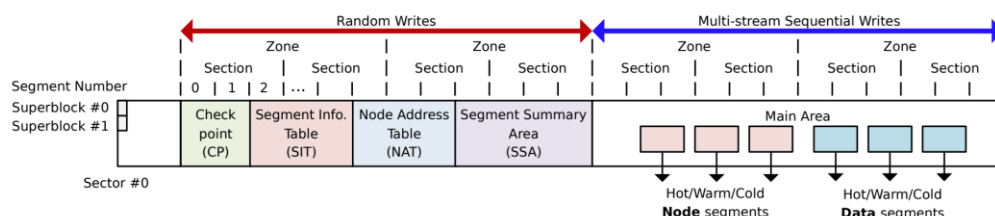


图 2-3 F2FS 磁盘布局与冷热数据分区(来源^[15])

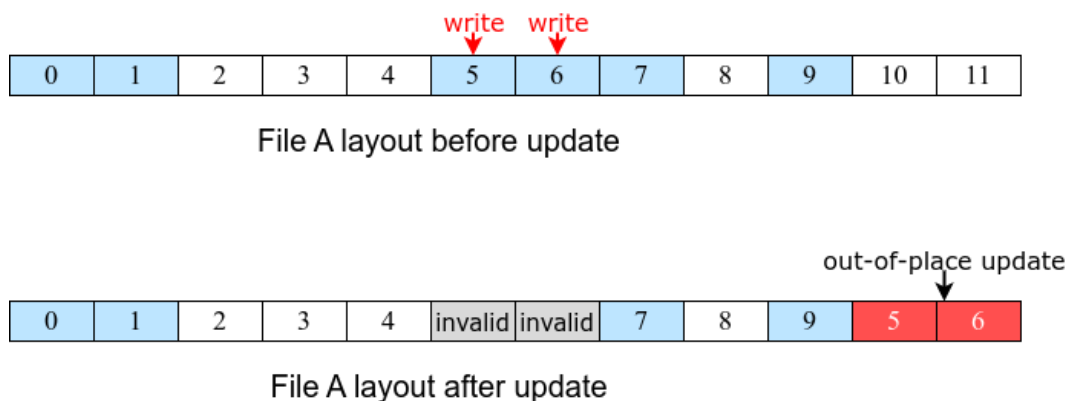


图 2-4 F2FS 异地更新过程示意

2.2.2 垃圾回收机制

异地更新会不断产生无效旧块，使同一段内有效块和无效块混杂。F2FS 通过垃圾回收（Garbage Collection, GC）回收这类空间。GC 通常选择有效块比例较低的 victim 段，读取其中仍然有效的数据块，将它们搬移到新的活跃日志段，并更新相应元数据，使旧段可以被擦除并重新投入分配。

GC 不是与普通 I/O 完全无关的后台过程。它需要识别 victim 段中仍然有效的文件数据块，重新读取并写入这些块，再修正逻辑块到新物理块的映射关系。因此，GC 路径同样会经过页缓存对象、逻辑块号、物理块号和写回提交等关键数据结构。

2.2.3 压缩文件机制

F2FS 还支持文件数据压缩。压缩以簇（cluster）为单位组织，一个簇由连续的若干文件系统块组成，簇大小由 `log_cluster_size` 参数决定。写入时，F2FS 将一个簇内的原始数据压缩后写入较少的物理块；读取时，F2FS 需要先读取压缩数据，再将其解压并填充回目标页缓存对象。

压缩路径在执行前会通过压缩上下文（`compress_ctx`）收集同一簇内的页缓存对象，并围绕簇边界完成压缩、解压和状态更新。因此，压缩文件路径与普通文件逐块映射路径不同，它不仅关心单个逻辑块的物理位置，还需要维护簇级组织关系和解压后的页缓存填充状态。

2.3 Large Folio（动态大页）机制

第一章中已提到动态大页可以表示更大粒度页缓存。其在 Linux 内核中的数据结构抽象是 `struct folio`。`struct folio` 最核心的数据成员为阶数（`order`）。阶数可以为 0，表示单个页，也可以大于 0，表示它是包含多个连续页一整个大页缓存对象。代码 2-1 展示了其核心数据字段。

代码 2-1 `struct folio` 数据结构简化定义

```
struct folio {  
    unsigned long flags;  
    struct address_space *mapping;  
    pgoff_t index;  
    void *private;  
    atomic_t _refcount;
```

```
struct page page;
};
```

上述结构中，mapping 和页描述符一样指向动态大页所属的地址空间，index 表示其在文件页缓存中的起始索引，private 可供文件系统保存私有状态。

动态大页的大小由阶数（order）决定。例如阶数为 0 时，其包含 1 个 4KB 页，阶数为 2 时包含 4 个 4KB 页，即 16KB^[6]。文件系统通过 mapping_set_large_folios 声明自身支持大粒度页缓存对象后，页缓存可以根据访问模式、预读窗口和内存分配情况选择合适阶数的动态大页。

因此，页缓存中可以存在不同阶数的动态大页，如图 2-5 所示。

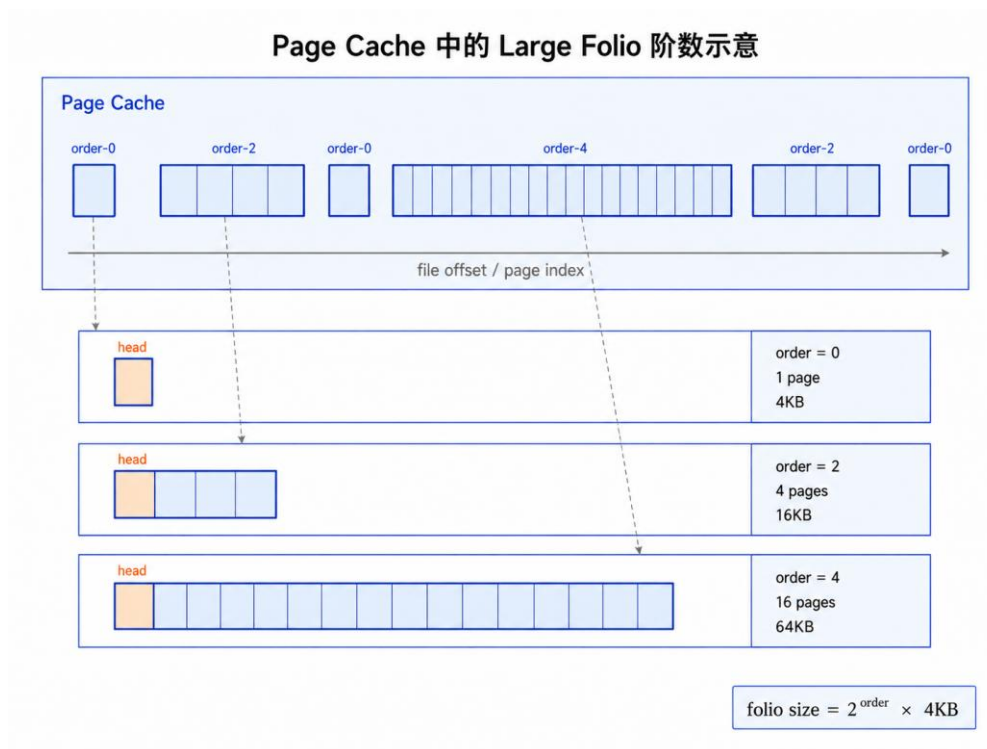


图 2-5 页缓存中的动态大页阶数示意

由于动态大页扩大的是页缓存对象粒度，并不自动改变文件系统块大小。以 64KB 动态大页为例，它在页缓存层表现为一个缓存对象，但在 F2FS 层仍对应 16 个 4KB 文件系统块。因此，文件系统仍需要提供覆盖该对象范围的块映射、子范围状态记录和 I/O 完成管理能力。

2.4 iomap 区间映射与动态大页状态管理

2.4.1 iomap 区间映射模型

第一章已经提到缓冲描述符在面对大块 I/O 时的局限性。本节则将详细介绍 iomap 整个框架组件以及其相对于缓冲描述符的优势。

iomap 框架使用区间化映射描述文件 I/O。其核心结构 `struct iomap` 描述文件中一段连续字节区间到设备字节地址的映射关系，包含文件内起始偏移 `offset`、区间长度 `length`、设备地址 `addr`、块设备指针 `bdev` 以及区间类型 `type`。由于 `length` 以字节为单位描述，iomap 可以用一个区间表示多个连续文件系统块，从而更适合与动态大页覆盖范围配合，其核心字段可简化表示如代码 2-2 所示。

代码 2-2 `struct iomap` 数据结构简化定义

```
struct iomap {
    u64      addr; /* disk offset of mapping, bytes */
    loff_t   offset; /* file offset of mapping, bytes */
    u64      length; /* length of mapping, bytes */
    u16      type; /* type of mapping */
    u16      flags; /* flags for mapping */
    struct block_device *bdev; /* block device for I/O */
    struct dax_device *dax_dev; /* dax_dev for dax operations */
    void     *inline_data;
    void     *private; /* filesystem private */
    u64      validity_cookie; /* used with .iomap_valid() */
};
```

图 2-6 展示了 iomap 与缓冲描述符在映射单元大小上的对比，直观地展现 iomap 的区间化优势。

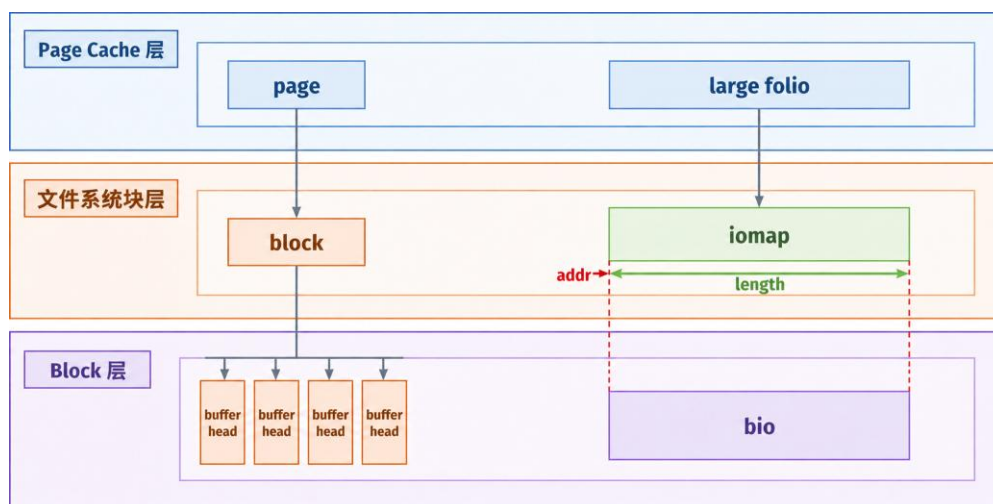


图 2-6 缓冲描述符与 iomap 映射模型对比

2.4.2 iomap 框架及迭代机制

iomap 通过迭代器模式驱动缓冲 I/O。文件系统通常提供开始钩子和结束钩子：前者根据给定文件偏移和长度填充 iomap 描述符，后者在一次区间处理结束后完成收尾。通用区间迭代器负责循环请求映射区间，并把每次得到的结果交给具体读写路径消费。

在缓冲读路径中，iomap 的预读入口或相关读取入口可以根据区间映射提交读 I/O；在缓冲写路径中，通用缓冲写入口可以在写入前请求映射或预分配；在写回路径中，通用写回入口负责根据脏区组织提交。不同路径共享区间迭代模型，但各自承担不同的页缓存状态更新和 I/O 提交职责。图 2-7 展现了 iomap 整个框架结构，来详细描述其迭代器模式如何运作：

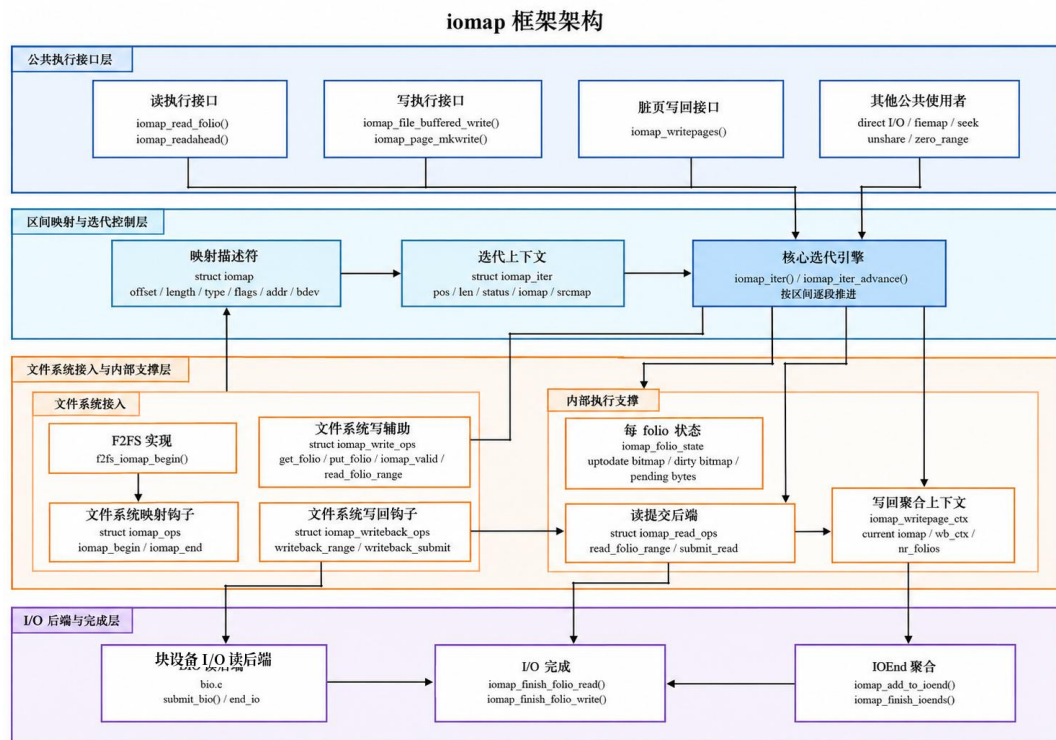


图 2-7 iomap 框架路径组织结构

如图所示，iomap 框架可以分为公共执行接口层、区间映射与迭代控制层、文件系统接入与内部支撑层以及 I/O 后端与完成层。公共执行接口层提供读执行接口、写执行接口、脏页写回接口和其他公共使用入口，这些接口面向不同文件系统读写路径复用。区间映射与迭代控制层以 struct iomap 描述一次文件区

间映射结果，以 `struct iomap_iter` 保存当前迭代位置、长度、状态和映射描述，并通过区间迭代入口与区间推进函数按照文件偏移逐段推进。文件系统接入层由文件系统映射开始钩子、映射结束钩子、写辅助钩子和写回钩子组成，F2FS 在这一层把自身块映射结果接入 `iomap` 的区间语义。内部支撑层维护动态大页的就绪状态位图、脏状态位图和 `pending bytes`，并连接读提交、写回聚合与完成路径。最下层的块设备 I/O 后端负责提交和完成处理，读完成、写完成以及 IOEnd 聚合最终反向更新页缓存状态。本文后续工作主要围绕文件系统接入层和内部支撑层展开，使 F2FS 的块映射、动态大页状态和块设备 I/O 完成过程能够衔接到 `iomap` 框架中。其中，`struct iomap_iter` 的核心字段可简化表示如代码 2-3 所示。

代码 2-3 `struct iomap_iter` 数据结构简化定义

```
struct iomap_iter {  
    struct inode *inode;  
    loff_t pos;  
    u64 len;  
    s64 processed;  
    unsigned flags;  
    struct iomap iomap;  
    struct iomap srcmap;  
    void *private;  
};
```

2.4.3 `iomap` 动态大页状态对象与子范围状态管理

在动态大页场景下，单个页缓存对象可能对应多个文件系统块，因此仅依赖对象级就绪、脏状态或写回状态标志无法表达内部子范围状态。`iomap` 为每个动态大页维护 `iomap` 动态大页状态对象，用于记录对象内部各个文件系统块的就绪状态、脏区状态以及异步 I/O 完成进度。其结构如图 2-8 所示。

图 2-7 iomap 动态大页状态对象中的逐块位图与 pending 计数

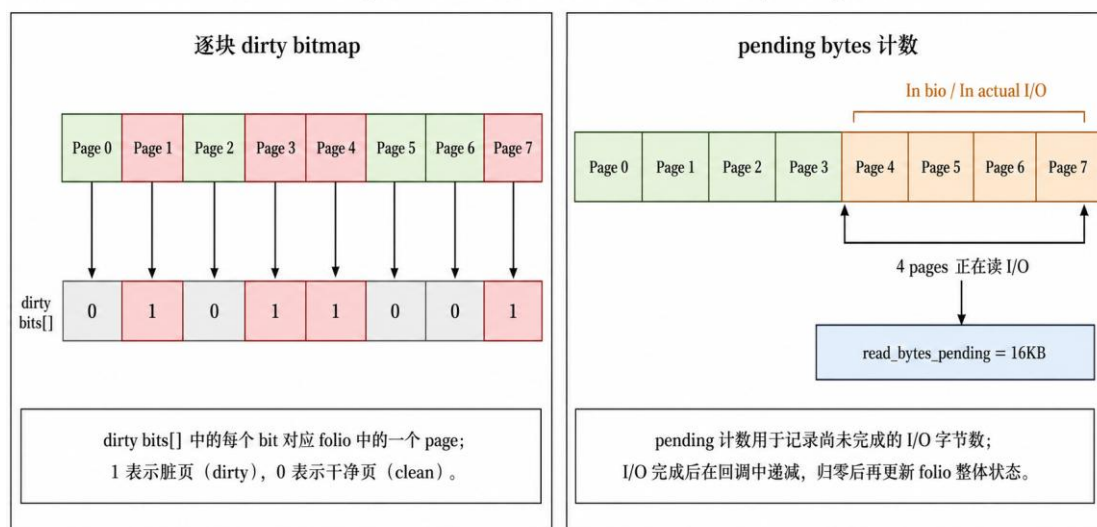


图 2-8 iomap 动态大页状态对象中的逐块位图与待完成计数

`iomap_folio_state` 主要包含两类信息：第一类是逐块位图，用于记录动态大页内部哪些子块已经就绪、哪些子块被标记为脏；第二类是待处理（pending）字节计数，用于统计当前动态大页上尚未完成的读或写 I/O 字节数。I/O 提交时增加相应待处理计数，完成钩子中减少计数，计数归零后再更新动态大页整体状态^[24]。表 2-2 汇总了 `iomap` 动态大页状态对象的主要数据成员的作用。

表 2-2 iomap 动态大页状态对象的主要数据成员

组件	作用	工作方式
uptodate 位图	记录动态大页内部哪些子块的数据已经就绪	以文件系统块为粒度置位和测试
dirty 位图	记录动态大页内部实际被修改的子范围	写入路径只标记覆盖写入范围对应的子块
read_bytes_pending	汇总尚未完成的读 I/O 字节数	提交读 I/O 时增加，完成钩子中减少，归零后解锁动态大页
write_bytes_pending	汇总尚未完成的写回 I/O 字节数	提交写回 I/O 时增加，完成钩子中减少，归零后结束 writeback

2.5 本章小结

本章全面介绍本文后续研究所需的基础知识。从用户态读写系统调用视角出发，从上至下分析一次缓冲 I/O 中，文件偏移、访问字节范围、页缓存对象、F2FS 块映射和块设备 I/O 请求之间的一一对应关系，辅以一个 64KB 缓冲读示例直观展示了完整流程。

在普通缓冲 I/O 路径之外，本章继续介绍 F2FS 面向闪存设备的特性。异地更新会改变文件修改后的实际落盘位置，垃圾回收会重新读取并搬移候选段中的有效块，压缩文件则以压缩簇为单位组织读写。这些特性都会参与文件数据位置和页缓存状态的维护，因此后续适配动态大页时都需要被纳入考虑。

最后，本章介绍动态大页和 iomap 两个本文核心概念。动态大页将页缓存对象从单个 4KB 页扩展到多个连续子页，因此可以覆盖更大的文件区间；iomap 则用字节区间描述文件逻辑范围到设备物理地址之间的映射关系，并基于迭代器框架自动完整大页 I/O 路径支持，并暴露公共执行接口使上层文件系统能直接复用。为后文设计 F2FS 动态大页适配方案和具体实现提供理论基础。

第三章 F2FS 动态大页缓冲 I/O 方案设计

3.1 设计目标与总体思路

前文已经说明，F2FS 原有缓冲 I/O 路径长期以 4KB 页缓存对象和 4KB 文件系统块为基本单位组织映射、提交与状态管理。动态大页扩大页缓存对象覆盖范围后，一个页缓存对象可能同时覆盖多个文件系统块，使原有页索引、逻辑块号、脏页状态和异步完成语义之间的对应关系发生变化。基于此，本文在不改变 F2FS 基本磁盘布局和通用页缓存框架的前提下，设计面向动态大页的缓冲 I/O 适配方案。

本文方案的设计目标是在不改变 F2FS 基本磁盘布局和通用页缓存框架的前提下，使 F2FS 能够为动态大页提供连续字节区间映射，并在统一状态对象中承载 F2FS 私有标志、子范围就绪/脏状态位图和异步完成计数，从而让普通读写、压缩文件与 GC 搬移路径在动态大页条件下保持一致的状态语义和完成判断。

为便于说明，后文主要以 64KB 动态大页为例展开分析；该示例对应一个动态大页覆盖 16 个 4KB 文件系统块的情况。

3.2 系统总体架构

本文设计的 F2FS 动态大页缓冲 I/O 方案以通用 iomap 框架为基础，在 F2FS 层提供区间映射、写入预分配和子范围写回等定制钩子，并进一步适配 F2FS 的 GC 与压缩文件路径。方案整体由普通文件区间化读写、扩展型动态大页状态对象、子范围写回控制、压缩文件适配和 GC 路径适配五部分组成，如图 3-1 所示。

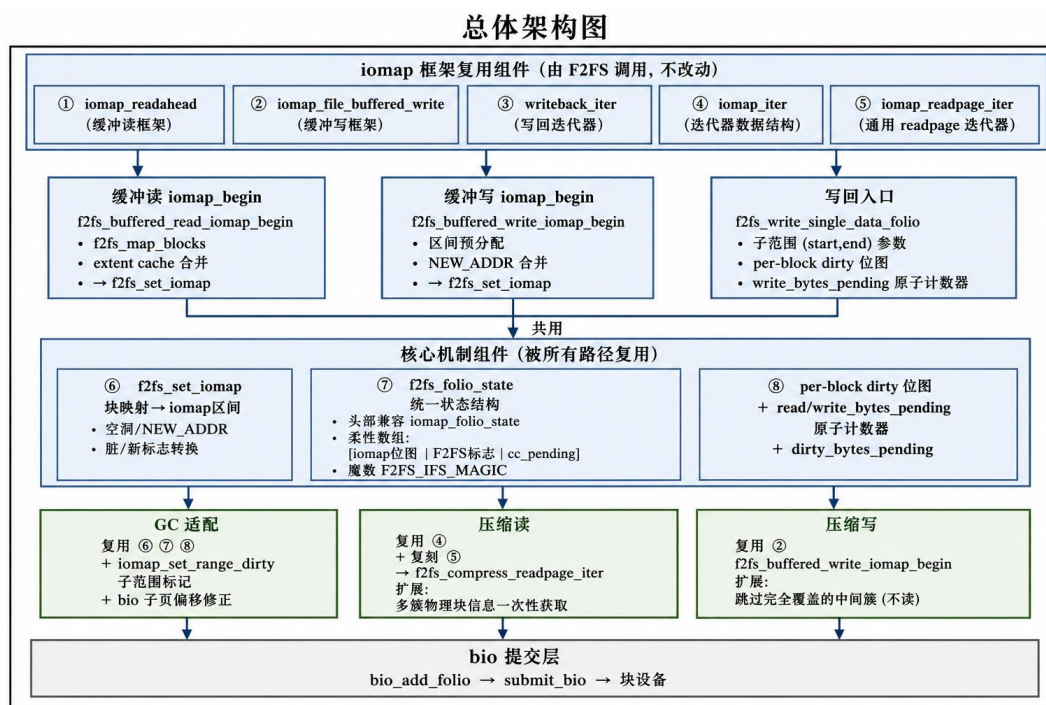


图 3-1 F2FS 动态大页缓冲 I/O 总体架构

如图 3-1 所示，整体方案可以从通用页缓存层、iomap 区间迭代层、F2FS 适配层和块设备 I/O 提交层理解。上层缓冲读写仍由页缓存和 iomap 负责调度，F2FS 在 iomap 开始钩子中将逻辑块映射转换为文件字节区间，并在写入准备阶段完成连续区间预留；动态大页状态对象位于页缓存对象与 F2FS 私有状态之间，用于保存子块就绪/脏状态位图、私有标志和读写完成计数；压缩文件与 GC 路径在进入块设备 I/O 前复用同一套状态对象，只在簇边界处理和有效块搬移定位上保留各自语义。这样的分层使普通读写、压缩处理、写回和后台搬移最终都围绕同一动态大页状态进行判断。

3.3 普通文件区间化映射机制设计

F2FS 原有普通文件读写路径以 4KB 文件系统块为基本单位查询映射关系。而根据第二章对动态大页的介绍，已经可知它能够表示多个连续的页，因此能够覆盖多个连续文件系统块。针对动态大页，需要以更大的字节范围推进 I/O，而 F2FS 原缓冲 I/O 路径只返回单块映射结果。由此，普通文件路径需要引入一个新的转换逻辑，能把一组逻辑块到物理块的映射结果整理成 iomap 描述的字节区间，而不再局限于单块推进读写。

该转换逻辑除去能够将块映射转化为字节区间, 还需要处理块映射结果本身的差异。因为同一段文件范围内, 逻辑块可以存在不同的块映射结构。逻辑块可以映射到具体的物理块地址, 也可能映射到未分配块、空洞块地址。不同类型的结果不能被简单合并到同一个 iomap 区间中, 而是需要划分成具有不同起始地址、长度和类型的区间。

基于这一需求, 本节设计统一的普通文件区间化映射函数。图 3-2 展示了上述整个块级映射结果转换为 iomap 字节区间全的过程和区间化映射函数发挥的作用:

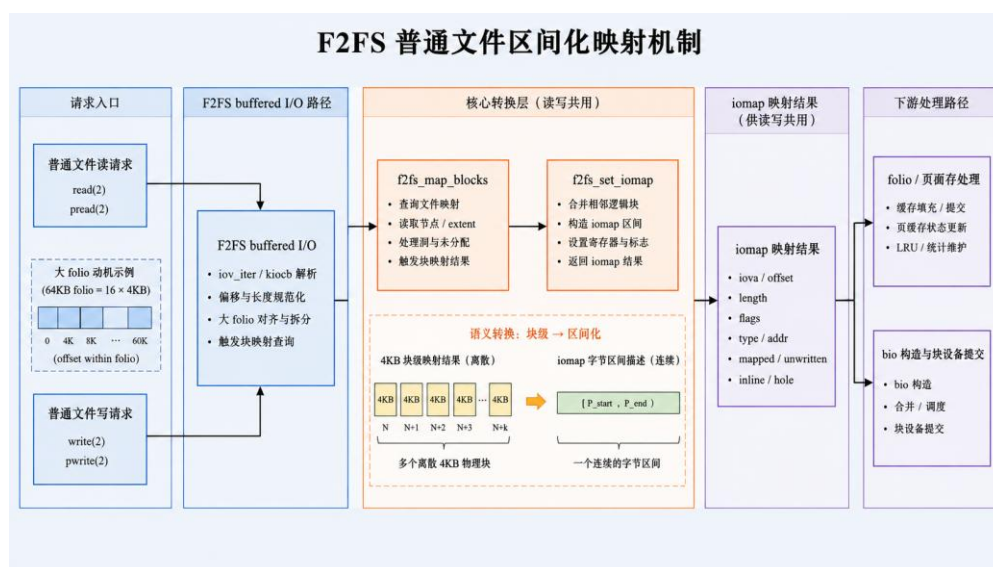


图 3-2 普通文件区间映射设计图

如图 3-2 所示, 本节设计的区间化映射函数一方面负责调用 F2FS 块映射逻辑, 将块级结果转换为 iomap 使用的字节区间; 另一方面集中处理不同映射结果类型之间的差异, 并返回成具备不同类型信息的区间。这样, 后续普通文件读写路径的 iomap 钩子函数可以通过同一个入口-获得稳定的区间描述, 动态大页内部的子范围状态维护和 I/O 提交也能围绕该区间结果继续展开。

3.4 扩展型动态大页状态对象设计

除映射粒度变化外, 动态大页还要求页缓存对象能够同时表达整对象状态和内部子块状态。然而 F2FS 原有私有字段用法与 iomap 动态大页状态对象之间会共同争夺使用同一个私有字段。F2FS 会直接利用该字段的低位保存迁移, 原子写等私有标志, iomap 缓冲 I/O 路径又要通过同一字段挂接 iomap 动态大页状态对象。

这会导致同一字段在不同路径中被分别解释为整数标志或状态对象指针，从而破坏跨路径状态语义，甚至导致字段被设置为整数标志时当成指针而错误解引用。因此，所有可能进入 `iomap` 路径的动态大页，不论阶数，都必须统一管理 F2FS 私有标志、`iomap` 位图状态和运行时计数。

第二章已经详细讲述过 `iomap` 动态大页状态对象会通过逐块位图记录内部子块是否就绪或为脏状态，并用 `pending` 字节计数汇总多个异步 I/O 完成事件。本文在此基础上设计 F2FS 扩展型动态大页状态对象：保留 `iomap` 状态对象头部布局，使通用 `iomap` 函数能够继续访问 `state_lock`、`read_bytes_pending` 和 `write_bytes_pending`，同时在柔性数组中扩展 F2FS 私有标志和压缩路径所需的待处理脏字节计数。其整体设计图如图 3-3 所示：

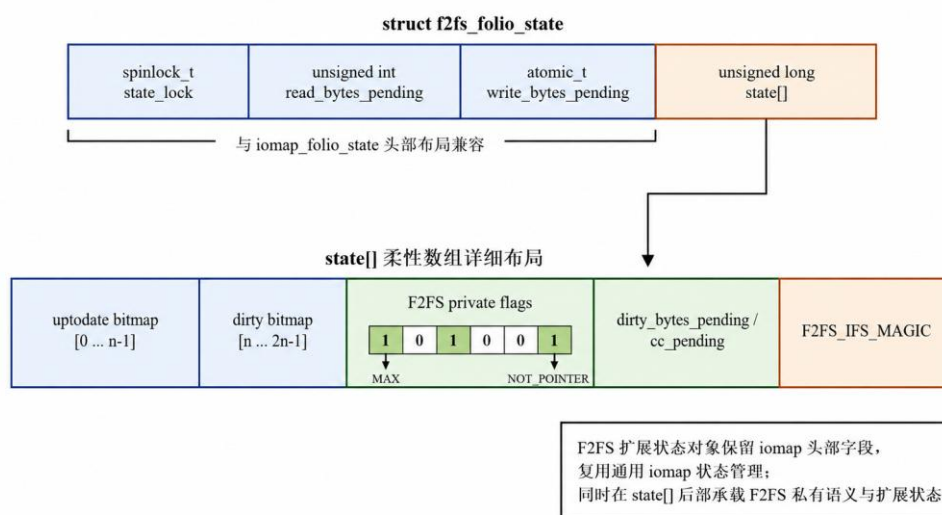


图 3-2 F2FS 扩展型动态大页状态对象布局

图 3-3 F2FS 扩展型动态大页状态对象布局

如图所示，F2FS 扩展型动态大页状态对象在内存布局上，前三个成员与 `iomap` 状态对象保持一致，而在功能上，柔性数组在 `iomap` 逐块位图之后额外保存 F2FS 私有标志和未完成脏字节计数，从而为 GC、原子写和压缩写回提供统一状态承载位置。同时，由于动态大页的私有字段可能指向原生 `iomap` 状态对象或 F2FS 扩展型状态对象，运行时需要识别对象类型。因此本文设计的 F2FS 扩展型动态大页状态对象会存储一个额外的魔数来和原生 `iomap` 状态对象形成区分。

3.5 子范围脏区与异步完成管理设计

动态大页进入写回路径后，脏页标记、文件逻辑块定位和 I/O 完成判断都需要从整页缓存对象粒度进一步细化。在 4KB 页缓存大小时代，一个页缓存对象对应一个文件系统逻辑块，页级脏标志可以直接表示该逻辑块需要写回，页缓存索引也可以作为后续块定位的依据。动态大页扩大页缓存对象后，一个 folio 内部可能只有部分 4KB 子块被修改，也可能在写回阶段对应多个异步块设备 I/O 请求。此时，整对象级脏标志只能说明该动态大页中存在脏数据，无法表达具体哪些子范围需要写回，也不能直接作为所有子 I/O 的完成边界。如果写回入口在开始阶段清除整个动态大页的脏标志，而缺少子块级脏状态记录，后续路径将无法区分实际修改范围，局部 4KB 修改可能被扩大为整个 64KB 动态大页写回。另一方面，简单将动态大页拆分为多个 4KB 单元后复用单页写回函数，也无法保证块索引正确推进；写回函数可能反复使用动态大页起始索引作为逻辑块位置，使动态大页内部后续子块无法写入对应文件范围。因此，写回适配需要同时解决子范围脏区记录、逻辑块定位和多 I/O 完成汇总问题。

基于上述需求，本文在扩展型动态大页状态对象中维护逐块脏状态位图，用于记录动态大页内部实际被修改的 4KB 子范围。整个设计思路图 3-4 所示：

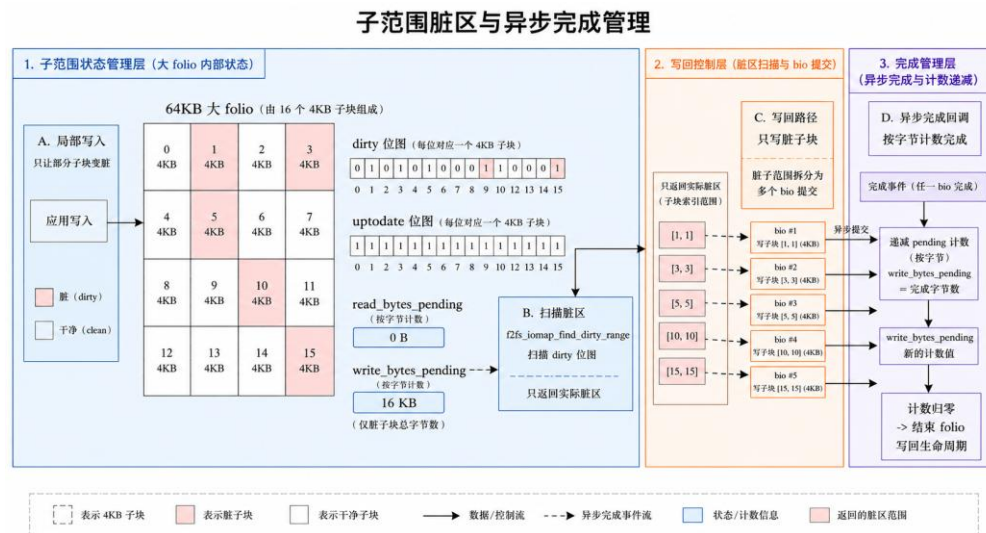


图 3-4 子范围脏区间与异步完成管理

如图所示，写回扫描时，会根据脏状态位图选择需要提交的子范围，并按照子块在动态大页内的相对位置计算对应文件逻辑块，避免将未修改子块纳入

写回。对于同一动态大页产生的多个异步块设备 I/O，状态对象通过写待处理字节计数记录还处于 I/O 之中的块设备请求，使动态大页只有在全部已提交子范围完成后才结束写回生命周期。局部失败或重试场景下，重试范围也被限制在失败子块内，已经成功完成的子范围不再重复写回。

3.6 压缩文件路径适配策略

动态大页进入压缩文件路径后，首先需要处理的是动态大页覆盖范围与压缩簇边界之间的不一致。F2FS 压缩文件以簇为数据组织单位，每个压缩簇由若干连续文件系统块组成，压缩、解压和写回都需要围绕簇边界进行。动态大页则以 folio 为页缓存对象，锁定、就绪状态、脏状态和 I/O 完成判断都围绕同一个页缓存对象展开。当一个动态大页跨越多个压缩簇，或者同时覆盖压缩区间与非压缩区间时，压缩路径既要保留簇级数据处理语义，又要保证动态大页生命周期能够统一结束。

在这种情况下，如果仅按照动态大页覆盖范围提交压缩 I/O，可能跨过压缩簇边界，破坏簇内数据的完整性；如果每个压缩簇仍独立维护锁定、提交和完成过程，又会在多个簇共享同一个动态大页时造成状态割裂。某个簇先完成时，动态大页内部其他簇可能仍处于解压、读写或回写过程中。因此，压缩文件路径需要在簇级处理边界和动态大页级状态管理之间建立转换关系。本文的压缩路径跨簇关系如图 3-5 所示。

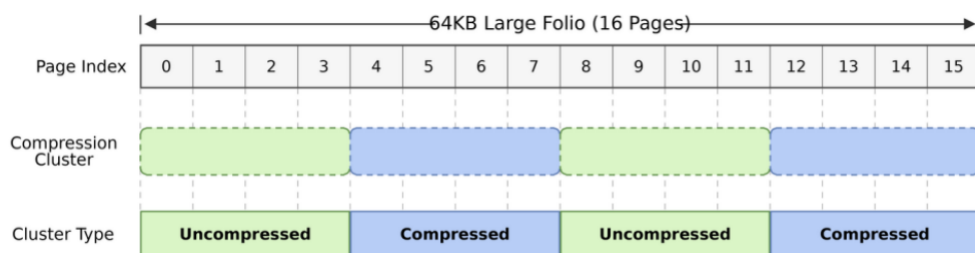


图 3-5 压缩路径中动态大页跨簇子区间示意

如图 3-5 所示，一个 64KB 动态大页覆盖 16 个 4KB 子块，其内部可能同时包含压缩簇和非压缩簇。对于非压缩区间，路径可以按照普通块映射方式提交块设备 I/O，对于压缩区间，则需要以压缩簇为单位获取物理块信息，并在解

压填充或压缩写回后更新动态大页内部对应子范围的状态。也就是说，压缩路径的入口面向动态大页覆盖范围，但实际数据处理仍然受压缩簇边界约束。

基于上述需求，本文在压缩读路径中设计簇感知的 `iomap_begin` 钩子和多簇压缩读页迭代逻辑。设计方案图如图 3-6 所示：

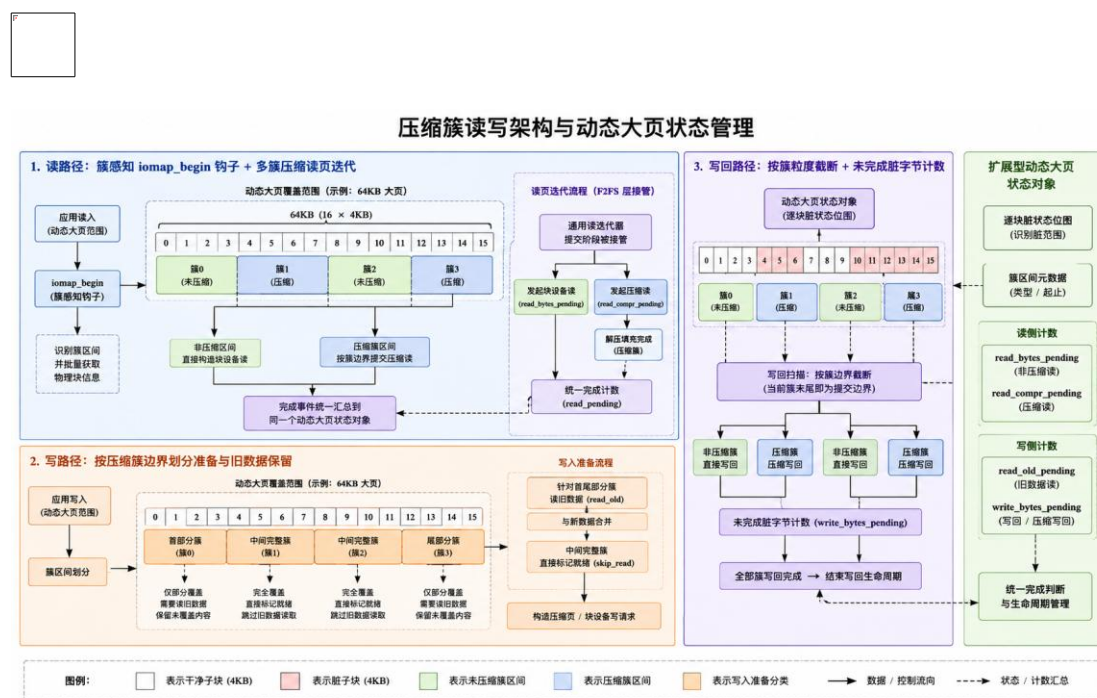


图 3-6 簇感知压缩读写大页架构图

如图所示，簇感知钩子根据动态大页覆盖范围识别其中涉及的压缩簇和非压缩簇，并在跨越多个簇时一次性获取相关簇的物理块信息。多簇压缩读页迭代逻辑在 F2FS 层接管通用读迭代器的提交过程：对非压缩簇直接构造块设备读请求，对压缩簇则按照簇边界提交压缩读，并将解压填充和直接磁盘读的完成事件统一汇总到同一个动态大页状态对象中。

压缩写路径同样以动态大页覆盖范围作为入口，但写入准备和旧数据保留需要按照压缩簇边界划分。本文将涉及的簇区间划分为首部部分簇、中间完整簇和尾部部分簇三类：首部部分簇只被本次写入覆盖一部分，需要读取旧数据以保留未覆盖内容；中间完整簇已经被写入范围完全覆盖，可以直接标记为就绪并跳过旧数据读取。通过这种划分，写路径既能适配动态大页带来的大范围写入，又不会破坏压缩簇内部的数据组织方式。

在压缩写回阶段，提交边界仍然需要回到簇粒度。即使动态大页状态对象中保存的是逐块脏状态位图，写回扫描时也不能简单把连续脏区整体提交，而是需要在当前压缩簇末尾截断写回范围。对于同一动态大页内尚未完成的压缩写回，扩展型动态大页状态对象通过未完成脏字节计数延迟解锁，使动态大页只有在相关簇的写回处理全部完成后才结束写回生命周期。这样，压缩路径可以在保持 F2FS 簇级语义的同时，将跨簇动态大页的状态维护和完成判断统一到同一套状态对象中。

3.7 GC 路径适配策略

GC 路径以单个逻辑块为搬移对象，而动态大页可能覆盖多个逻辑块。因此，GC 适配需要保留单块搬移语义，并在动态大页内部定位目标子范围。

在 GC 路径中，后台搬移仍以单个逻辑块为对象，所以需要针对动态大页内部只标记目标逻辑块对应的 4KB 子范围为脏状态；GC 私有标志通过扩展型状态对象保存，不再重新占用动态大页私有字段；读写块设备 I/O 请求构建时则显式计算子页在动态大页内的相对偏移，使请求范围只覆盖目标子块。本文的 GC 子块级标记设计方式如图 3-6 所示。

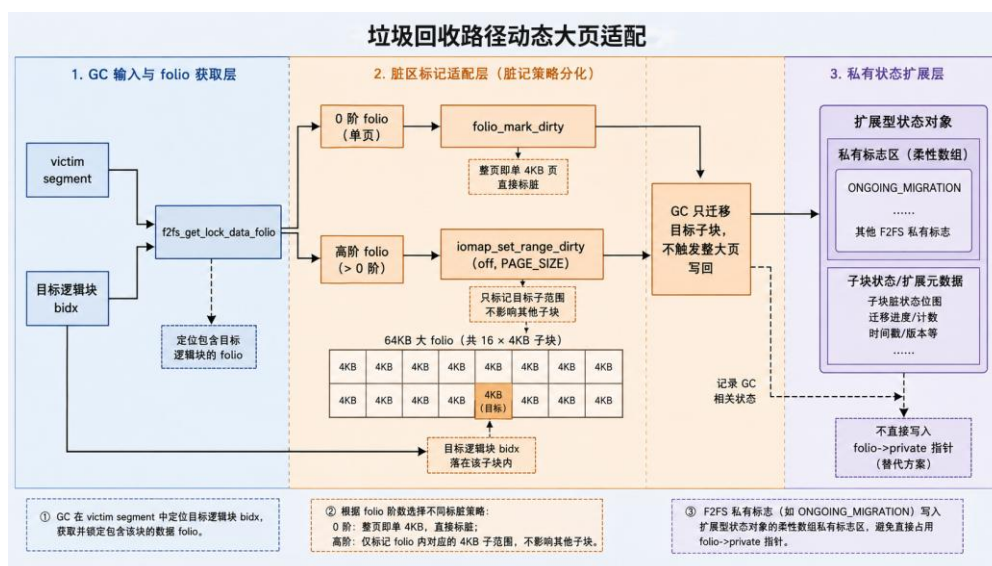


图 3-6 垃圾回收大页适配

如图 3-6 所示，GC 路径首先根据 victim segment 中的逻辑块信息获取对应 folio。对于 0 阶 folio，目标块与页缓存对象仍然一一对应，可以沿用原有标记方式；对于高阶 folio，路径需要根据目标逻辑块计算其在动态大页内的相对偏

移，只将对应 4KB 子范围标记为脏，而不是把整个 folio 标记为脏。这样，后续写回扫描时只会选择该目标子块作为提交范围。

GC 相关私有标志统一保存到 F2FS 扩展型动态大页状态对象中，与子块脏状态、就绪状态和完成计数放在同一对象内管理。构建块设备 I/O 请求时，路径继续以单个逻辑块为搬移对象，但根据子块偏移确定动态大页内的访问范围，使请求只覆盖目标 4KB 子块。通过这种方式，GC 路径既保持原有单块搬移语义，又避免动态大页扩大搬移范围或破坏 folio 私有字段的状态含义。

3.8 本章小结

本章给出了 F2FS 动态大页缓冲 I/O 的总体方案。围绕动态大页进入 F2FS 后引发的映射粒度、状态承载、写回范围、压缩簇边界和 GC 子块定位问题，本文设计了区间化映射、扩展型动态大页状态对象、子范围写回、压缩文件路径适配和 GC 路径适配机制。

上述机制以 F2FS 区间设置函数和 F2FS 扩展型动态大页状态对象为基础：前者提供块到字节区间的统一转换，后者提供跨普通文件、压缩文件、写回和 GC 路径的统一状态承载。第四章将在此基础上说明各机制在 F2FS 具体函数、算法和路径中的实现方式。

第四章 F2FS 动态大页缓冲 I/O 实现

本章对应第三章提出的总体 F2FS 动态大页缓冲 I/O 方案设计，会根据每个具体模块的子方案设计，逐个讲解具体实现。

4.1 普通文件区间化读写实现

普通文件路径是动态大页缓冲 I/O 最基础的入口，也是区间化映射、状态维护和完成钩子协同工作的主线。本节先说明读写钩子如何把页缓存覆盖范围转换为 iomap 所需的字节区间，再说明写入准备阶段怎样把连续映射和空间预留统一到同一套处理流程中。

4.1.1 块映射到 iomap 区间的转换实现

F2FS 区间设置函数承担的是块级映射到字节区间的语义转换，因此它既要识别连续映射，也要处理空洞和延迟分配这类边界状态。该函数的实现方式直接决定后续 iomap 迭代是否能够保持连续推进。

下面给出伪代码 4-1，用于说明 F2FS 区间设置函数如何把块映射结果转换为 iomap 字节区间，并处理空洞、延迟分配与连续映射等边界状态。

伪代码 4-1 f2fs_set_iomap

```
Input: inode, map, offset, length, dio

Output: iomap

iomap.offset <- F2FS_BLK_TO_BYTES(map.m_lblk)    // 逻辑块转字节

if map has F2FS_MAP_MAPPED:    // 如果是有效映射

    iomap.length <- F2FS_BLK_TO_BYTES(map.m_len)    // 物理块长度转字节长度

    if dio and map.m_pblk == NEW_ADDR:

        return -EINVAL

    if map.m_pblk != NEW_ADDR:

        iomap.type <- IOMAP_MAPPED; set IOMAP_F_MERGED

    iomap.addr <- F2FS_BLK_TO_BYTES(map.m_pblk)    // 物理块转字节地址

else:

    iomap.type <- IOMAP_UNWRITTEN; iomap.addr <- IOMAP_NULL_ADDR    // 未写区间

else:

    if dio and write flag set:
```

```

return -ENOTBLK

if map.m_pblk == NULL_ADDR:    // 处理空洞
    iomap.length <- hole_end(map) - iomap.offset
    iomap.type <- IOMAP_HOLE; iomap.addr <- IOMAP_NULL_ADDR    // 空洞区间
elseif map.m_pblk == NEW_ADDR: // 处理预分配
    iomap.length <- F2FS_BLK_TO_BYTES(map.m_len)    // 预分配块长度转字节
    iomap.type <- IOMAP_UNWRITTEN; iomap.addr <- IOMAP_NULL_ADDR    // 未写
    区间
else:
    return -EFAULT

if map has F2FS_MAP_NEW:
    set IOMAP_F_NEW

if inode is datasync-dirty or offset + length > i_size(inode):
    set IOMAP_F_DIRTY

return 0

```

该函数主要区分三类映射结果。若块映射已经命中有效物理块，则生成已映射区间，并在连续映射场景下写入合并标志；若物理块地址表示新分配但尚未落盘的地址，则生成未写区间。若当前范围没有有效物理块，则继续区分两类空洞：没有直接节点条目时，空洞终点以下一个存在节点块的逻辑页号为界；已有节点条目但映射为空时，则以当前映射长度为界。最后，新分配状态和索引节点脏状态会分别转换为 iomap 的新建标志和脏标志。

在实现中，上层 iomap 开始钩子将动态大页覆盖的完整区间一次性传递给 F2FS 块映射函数。块映射函数内部的块合并判断逻辑会向后合并物理地址和逻辑地址都连续的相邻块，并把连续映射长度扩展为可直接返回的子区间长度；F2FS 区间设置函数再将该结果转换为 iomap 字节区间。以 64KB 动态大页为例，当 16 个 4KB 块连续映射时，该路径可以通过一次区间映射完成，从而减少重复映射查询和 dnode 锁竞争。

4.1.2 普通文件缓冲读钩子实现

普通文件缓冲读钩子以动态大页覆盖的字节范围为输入，先计算对应的逻辑块区间，再调用 F2FS 块映射函数获取连续映射结果，并最终交由 F2FS 区间

设置函数填充 `iomap` 描述符。该钩子把相邻块合并到同一次区间处理里，减少重复查找和不必要的块级遍历。

下面给出伪代码 4-2，用于说明缓冲读钩子如何把 `offset` 和 `length` 转换为逻辑块区间，并把连续映射结果回填为 `iomap` 描述符。

伪代码 4-2 f2fs_buffered_read_iomap_begin

```

Input: inode, offset, length, flags

Output: iomap

map.m_lblk <- F2FS_BYTES_TO_BLK(offset)

map.m_len <- F2FS_BYTES_TO_BLK(offset + length - 1) - map.m_lblk + 1

map.m_next_pgofs <- &next_pgofs; map.m_may_create <- false

if flags has IOMAP_WRITE:

    return -EINVAL

err <- call f2fs_map_blocks(inode, &map, F2FS_GET_BLOCK_IOMAP)

if err != 0:

    return err

if map.m_pblk == COMPRESS_ADDR:    // 压缩簇分流

    return -EINVAL

return call f2fs_set_iomap(inode, &map, iomap, flags, offset, length,
false)

```

该钩子的先将字节范围转换为逻辑块范围，再在块映射函数返回后，通过 4.1.1 节的区间设置过程把映射结果写入 `iomap` 描述符。在传入钩子函数的参数中，`iomap` 模式标志用于告诉块映射函数当前调用来自 `iomap` 路径。块映射函数随后在目标逻辑块范围内向后遍历，并将逻辑地址和物理地址都连续的相邻块合并到同一映射结果中。每当块映射函数成功合并了一个文件系统块，连续映射长度就相应增加，遍历结束后，合并后的连续区间信息会被缓存，以供后续请求直接命中。

区间设置完成后，`iomap` 框架迭代器按照区间长度更新当前位置，读页迭代器将把当前区间内的页面加入块设备 I/O 请求，同时维护就绪位图，并在整个动态大页的所有子区间全部就绪后解锁，自动完成全部动态大页到块设备 I/O 的推进任务。文件系统因此不需要在该钩子中手动处理块设备 I/O 请求构建、

完成钩子注册或解锁时机。原生读路径与本文区间化读路径在映射粒度、持锁次数和设备提交次数上的差异在表 4-1 进行汇总，详细展现了本文读路径在削减映射开销上的优势：

表 4-1 原生度路径与本文实现读路径对比

指标	原生读路径	本文实现读路径
F2FS 块映射 调用次数	16 次（逐块）	1 次（连续区间）或 N 次（N=连续子区间数）
映射请求粒度	每次 1 块	一次整个字节区间
块合并	不适用（每次只请求 1 块）	块合并判断 遍历中自动合并
dnode 锁操作	16 次	1 次
块设备 I/O 请求提交	16 次独立提交	iomap 框架自动合并为更少提交

4.1.3 普通文件缓冲写钩子实现

普通文件缓冲写钩子的核心任务是在写入前完成目标区间的映射查询和空间预留。动态大页覆盖多个文件系统块后，写入准备阶段不能再按单块反复分配，而需要尽量以连续区间为单位完成 dnode 查找、段锁获取和新块预留，使后续写入路径能够在同一套 iomap 区间语义下推进。

伪代码 4-3 f2fs_buffered_write_iomap_begin: range allocation for buffered write

```

Input: inode, offset, length, flags

Output: iomap

map.m_lblk <- F2FS_BYTES_TO_BLK(offset)

map.m_len <- F2FS_BYTES_TO_BLK(offset + length - 1) - map.m_lblk + 1

map.m_may_create <- true

iomap.bdev <- inode.sb.bdev; iomap.folio_ops <- f2fs_iomap_folio_ops

err <- call f2fs_map_blocks(inode, &map, F2FS_GET_BLOCK_IOMAP)

if err != 0 and err != -ENOSPC:

    return err

if map.m_pblk == NULL_ADDR:    // 空洞处理

    err <- call f2fs_map_blocks_preallocate(inode, &map)    // 空洞预分配

    if err != 0 and err != -ENOSPC:

        return err
    
```

```
return call f2fs_set_iomap(inode, &map, iomap, flags, offset, length,
false) // 物理块转字节区间映射
```

和缓冲读 `iomap` 钩子函数类似，该钩子函数同样分成两步，第一步仍然以 `iomap` 模式调用 F2FS 块映射函数，获取物理块映射信息。第二步则有所不同，钩子函数内部会检测上次块映射信息是否是预分配地址块，只有在这种情况下，会继续遍历到直接节点页末尾，并批量预留新块。缓冲写钩子函数最重要的优化点是一次预留可以覆盖整个写入区间同时返回多个逻辑块长度。此时物理地址暂时标记为新地址的统一映射。

批量预分配能生效的关键在于块合并判断逻辑对新地址的特殊处理：当当前块和前一映射都处于新分配但尚未落盘的状态时，合并条件同样成立。这使连续新分配区间能够作为一个大区间返回，而不会退化为每次只处理一个块的细粒度分配。

由于缓冲写钩子函数为 `iomap` 框架缓冲写接口服务，该写路径能共享到 `iomap` 动态大页状态对象的脏标记位图，只把本次真正修改的字节区间标记为脏。同时，文件系统钩子在会在动态大页已经锁定的上下文中执行，使调用者不需要手动处理并发截断竞争。

表 4-2 汇总了 F2FS 原生缓冲写路径与本文实现的写路径的具体对比：

表 4-2 原生逐块写准备路径与本文实现区间预分配路径对比

指标	原生写路径	本文实现写路径
空间分配	逐块分配，1 次 1 块	区间预分配，1 次 N 块
<code>dnode</code> 查找	N 次	1 次
F2FS 预留新块	N 次	1 次（内部批量）
连续块合并	逐块返回，无法合并	合并连续 <code>NEW_ADDR</code>
原子文件	单独复制 <code>dnode</code> 逻辑	复用映射逻辑，仅切换 <code>dnode</code>

4.2 扩展型动态大页状态对象实现

扩展型动态大页状态对象是本文把 F2FS 私有状态接入 `iomap` 的基础，它既要兼容旧式私有标志用法，也要支持高阶动态大页下的子范围位图和完成计数。本节围绕对象分配、迁移与标志访问三个层面展开。

下面给出伪代码 4-4，用于说明状态对象在私有字段为空、已有旧式标志和已挂接状态对象三种情况下的分配与迁移处理。

伪代码 4-4 f2fs_ifs_alloc: attach or migrate state

```

Input: folio, gfp, force_alloc

Output: f2fs_folio_state or NULL

old <- folio_get_private(folio)

if folio_order(folio) == 0 and not force_alloc: // 0 阶 folio 直接退出
return NULL

if old != NULL:

rbp <- READ_ONCE(old.read_bytes_pending)

if rbp >= F2FS_IFS_MAGIC:

return old

new <- call kmalloc(sizeof(f2fs_folio_state) +
f2fs_ifs_total longs(folio))

if new == NULL:

return NULL

call ifs_to_f2fs_ifs(old, new, folio)    // 迁移旧状态

init counters and state_lock

call folio_change_private(folio, new)

call kfree(old)

return new

new <- call kzalloc(sizeof(f2fs_folio_state) +
f2fs_ifs_total longs(folio))

if new == NULL:

return NULL

init counters and state_lock

if folio_test_uptodate(folio): // 整 folio 就绪标志位存在,置位全就绪位图

set all uptodate bits

if folio_test_dirty(folio): // 整 folio 脏标志位存在,置位全就脏块位图

set all dirty bits

call folio_attach_private(folio, new)

return new

```

第一种，动态大页的私有字段为空：直接分配并初始化新的 F2FS 扩展型动态大页状态对象，根据当前动态大页的就绪/脏状态 状态初始化对应位图。第二种，动态大页的私有字段已指向 iomap 动态大页状态对象（魔数不匹配）：分配更大的 F2FS 扩展型动态大页状态对象，将旧的逐块位图内容通过 F2FS 状态转换函数拷贝到新结构体对应位置，写入魔数，通过动态大页字段替换函数原子替换指针并释放旧结构体。第三种，魔数匹配：直接返回已有指针。

为兼容 F2FS 原有的大量私有标志设置调用点，本文设计了统一的私有标志访问宏族。以获取操作为例：首先检查动态大页的私有字段是否已设置“非指针”标志；若是，说明当前字段存储的是旧式直接标志位，直接在该值上进行位操作（快速路径，兼容元数据动态大页，若否，说明存储的是指针，再通过魔数校验访问柔性数组中的 F2FS 私有标志区域。设置操作的分配策略通过“当前动态大页是否可能进入 iomap 缓冲写路径”来判断：若可能进入该路径，即使该动态大页当前为 0 阶且尚未分配任何私有状态，也必须先分配状态对象。对应关系如图 4-1 所示。

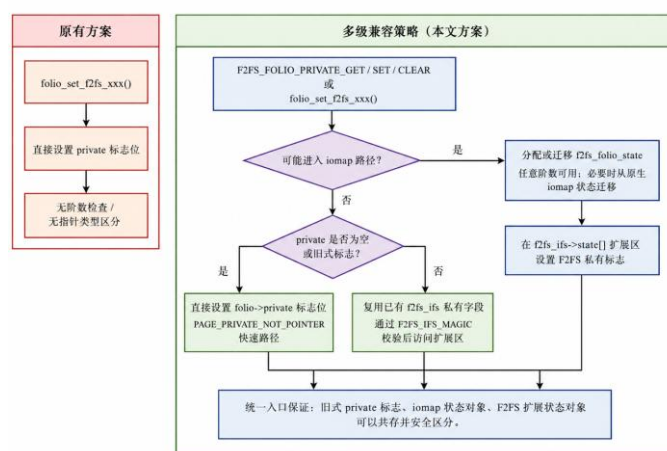


图 4-1 F2FS 私有字段兼容策略

4.3 压缩文件动态大页读写实现

压缩文件路径比普通文件路径更复杂，因为它同时受到簇边界、解压流程和完成计数的约束。本节先说明压缩读写钩子怎样在 iomap 框架下保留簇语义，再说明读写过程如何借助统一状态对象维持正确的就绪与解锁时机。

4.3.1 压缩文件缓冲读实现

压缩读路径需要同时处理多簇映射、压缩簇提交和动态大页完成计数。与普通文件读路径相比，它不能在单个簇完成后立即结束动态大页生命周期，而需要把解压填充和非压缩区间的直接读完成统一汇总。

下面给出伪代码 4-5，用于说明压缩读钩子如何把单簇入口扩展为多簇映射，并在 iomap 迭代中保持连续区间长度。

伪代码 4-5 f2fs_buffered_read_compress_iomap_begin

```

Input: inode, pos, length, flags

Output: iomap

ctx <- iter.private

index <- pos >> PAGE_SHIFT

folio <- ctx.cur_folio or call readahead_folio(ctx.rac)

cluster <- cluster_idx(ctx.cc, index)

all <- (folio_nr_pages(folio) - (index - folio.index)) >>
ctx.cc.log_cluster_size

iomap.offset <- F2FS_BLK_TO_BYTES(F2FS_BYTES_TO_BLK(pos))

iomap.bdev <- inode.sb.bdev

if index != cluster << log_cluster_size or all <= 1:    // 非多簇路径
err <- call f2fs_get_dnode_of_data(ctx.dn, cluster << log_cluster_size)

if err != 0:

return err

if data_blkaddr(ctx.dn) == COMPRESS_ADDR:

ctx.compressed_clusters <- 1

iomap.length <- min(length, cluster_end(index) - index)

iomap.type <- IOMAP_MAPPED

return 0

else:

all <- min(all, MAX_PBLKS_INFO_SIZE)

for i <- 0 to all - 1:    // 批量扫描簇

cidx <- cluster + i

if call f2fs_lookup_read_extent_cache(inode, cidx, ei):

```

```

ctx.pblks_arr[i] <- extent pblk info

else:

err <- call f2fs_get_dnode_of_data(ctx.dn, cidx << log_cluster_size)

if err != 0:

return err

if data_blkaddr(ctx.dn) != COMPRESS_ADDR:    // 碰到非压缩簇, 回退普通路径
break

ctx.pblks_arr[i] <- dnode pblk list

process++

ctx.compressed_clusters <- process    // 记录簇数量

iomap.length <- bytes(process clusters); iomap.type <- IOMAP_MAPPED

return 0

return call f2fs_buffered_read_iomap_begin(inode, pos, length, flags,
iomap)

```

在旧的单簇路径中, 映射长度会被截断到当前位置至当前簇末尾的范围, 外层循环每次迭代最多覆盖一个压缩簇。新的多簇路径在检测到当前动态大页对齐于簇边界且跨越多个簇时, 会遍历这些簇并从 `dnode` 中取得各簇的物理块地址, 再把映射长度扩展为覆盖所有连续压缩簇的字节总长度。这样外层 `iomap` 区间迭代器一次即可覆盖多个簇的物理范围, 减少迭代次数和映射查询开销。

下面给出伪代码 4-6, 用于说明压缩读页迭代器如何复用 `iomap` 的外层推进逻辑, 并在内部切换为适配压缩簇的提交策略。

伪代码 4-6 `f2fs_compress_readpage_iter`

```

Input: iter, ctx

Output: advanced length or error

pos <- iter.pos

length <- iomap_length(iter)

folio <- ctx.cur_folio

fifs <- call f2fs_ifs_alloc(folio, iter.flags, false)

```

```

call iomap_adjust_read_range(iter.inode, folio, &pos, length, &poff,
&plen)

if plen == 0:

goto done

if call iomap_block_needs_zeroing(iter, pos):

call zero_and_mark_uptodate(folio, poff, plen)

goto done

if ctx.compressed_clusters > 0:    // 压缩簇批量读

ret <- call f2fs_do_read_multi_folios(ctx, pos, plen)

else:

ret <- call f2fs_do_read_single_folio_iomap(iter, ctx, pos, plen, poff)

if ret != 0:

return ret

done:

length <- pos - iter.pos + plen

return call iomap_iter_advance(iter, &length)

```

该函数保留了 `iomap` 通用读页迭代器的外层推进方式，仍由 `iomap` 负责计算实际读取范围、处理空洞填零并推进当前区间。不同之处在于，F2FS 在内部提交阶段根据当前区间是否属于压缩簇选择不同路径：压缩区间先进入压缩上下文，等待簇内页面聚合完成后统一读取并解压；非压缩区间则直接构建块设备 I/O 请求并提交。

之所以在 F2FS 层定制迭代器，是因为压缩数据具有延迟提交语义。通用 `iomap` 读页迭代器会在确定读取范围后立即构建并提交块设备 I/O 请求，而压缩路径要求页缓存对象先在压缩上下文中聚合为簇，待簇满或预读结束后再提交读取。如果直接修改通用 `iomap` 代码，会影响其他文件系统的读路径，因此本文只在 F2FS 层实现外层推进方式一致、内部提交策略可分发的压缩读迭代器。

两条 I/O 路径的完成钩子最终都会汇集到 F2FS 动态大页读完成处理过程。每完成一个子区间，扩展型状态对象中的未完成读字节计数都会扣减相应长度；只有当计数归零，或仅剩用于区分状态对象的魔数偏置时，动态大页才会被统一解锁并结束读生命周期。对于一个跨越两个压缩簇和一个非压缩区间的

64KB 动态大页，初始计数会累加所有子区间的字节数，三条路径分别完成后各自扣减对应长度，直到全部完成后才触发动态大页读结束处理。

4.3.2 压缩文件缓冲写实现

压缩文件写路径的难点不在于写入本身，而在于写入前如何补齐首簇、中间完整簇和尾簇的旧数据，并保持压缩上下文要求的就绪顺序。

下面给出伪代码 4-7，用于说明压缩写入准备如何围绕头、中、尾三类簇完成旧数据检查和就绪标记。

伪代码 4-7 f2fs_buffered_write_iomap_begin: compressed range prepare

```

Input: inode, offset, length, flags

Output: iomap with prepared folio

folio <- call iomap_get_folio(iter, offset, length)
fifs <- call f2fs_ifs_alloc(folio, iter.flags, false)
if folio_order(folio) > 0 and fifs != NULL:
    fifs.read_bytes_pending <- fifs.read_bytes_pending + 1
head <- first cluster touched by offset
tail <- last cluster covered by folio
if head is partial:
    prepare head range; add pending bytes
if any page is stale:
    call f2fs_read_multi_folios(cc, pblks, dn, bio)
    call f2fs_reset_compress_ctx(cc)
for c <- first_full_cluster to last_full_cluster:    // 跳过完整覆盖簇读入
    err <- call f2fs_ensure_dn_for_cluster(inode, c, dn)
    if err != 0:
        return err
    if data_blkaddr(dn) != COMPRESS_ADDR:    // 回退普通路径
        goto normal_path_or_done
    call mark_full_cluster_uptodate(folio, c)    // 整簇置为就绪
    iomap.length <- iomap.length + cluster_size
if tail is partial and tail != head:

```

```
prepare tail range; add pending bytes

if any page is stale:

call f2fs_read_multi_folios(cc, pblks, dn, bio)

if bio != NULL:

call f2fs_submit_read_bio(sbi, bio, DATA)

while fifs.read_bytes_pending != F2FS_IFS_MAGIC + 1:

call f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT)

fifs.read_bytes_pending <- fifs.read_bytes_pending - 1

call folio_mark_uptodate(folio)

iomap.private <- folio; iomap.type <- IOMAP_MAPPED

return 0
```

为避免异步读取提前触发动态大页解锁，写入准备阶段在提交旧数据读取前先向未完成读字节计数加入一个临时偏置。该偏置保证完成钩子即使先返回，也不会直接结束动态大页读状态。随后 `iomap` 开始钩子显式等待计数降至只剩魔数和临时偏置的状态，表示所有旧数据读取已经完成；等待结束后再移除偏置，动态大页由当前持有方继续控制锁的释放。

写入准备过程分为三个阶段。第一阶段处理起始位置不对齐簇边界的首簇：如果簇内所有子页已经就绪，则直接跳过读取；如果存在未就绪子页，则发起磁盘读取补齐旧数据。第二阶段处理中间完整簇，这些簇会被本次写入整体覆盖，旧数据没有保留价值，因此直接把对应动态大本子范围标记为就绪，跳过磁盘读取。第三阶段处理尾部边界不对齐的簇，逻辑与首簇相同。对于非压缩区域，该流程回退到普通文件路径。

执行阶段，F2FS 获取动态大页过程会先检查准备阶段是否已经挂接了可直接使用的动态大页；若已经挂接，则直接返回这个已经完成旧数据填充的对象；否则回退到通用 `iomap` 的动态大页获取流程，按标准方式分配并加锁。这样，`iomap` 核心框架在真正写入数据时，拿到的动态大页已经在开始钩子中完成必要的压缩数据读取和同步等待。

原有压缩写路径通常受压缩簇大小限制，跨簇写入需要拆成多次准备操作。本文在动态大页范围内统一组织写入准备，一次分配覆盖全部相关簇的动态大页，并在单次 `iomap` 开始钩子中完成必要簇的数据准备。

4.4 动态大页子范围写回实现

依照第三章子范围写回设计，将脏区收敛到真正需要回写的子块上，是本文降低写放大最关键环节。本节将先对这点进行说明，随后将继续说明 I/O 完成计数和重试收敛，最后，再说明压缩文件写回如何在簇边界和延迟解锁约束下复用同一核心循环。

4.4.1 普通文件子范围写回实现

普通文件写回的目标，是把整动态大页的写回粒度降到实际变脏的子块粒度，并让提交、完成和重试都围绕子范围展开。这样既能减小写放大，也能让完成计数和状态回收更精确。

下面给出伪代码 4-8，用于说明单动态大页写回如何依据 脏状态 位图定位脏子块，并按字节范围驱动提交与完成计数。

伪代码 4-8 `f2fs_write_single_data_folio`

```

Input: folio, wbc, compr_blocks, from_compress, start, end

Output: err, submitted_pages_count

if folio_order(folio) == 0:

    err <- call f2fs_write_single_data_page(folio, wbc)

    if err == -EAGAIN and sync writeback:

        call f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT)

        goto retry_single

    return err

fifs <- folio.private

first <- (start - folio_pos(folio)) >> PAGE_SHIFT // 计算真正写回区间

last <- (end - 1 - folio_pos(folio)) >> PAGE_SHIFT

for i <- first to last:    // 子页范围写回

    page <- folio_page(folio, i)

    fio <- build_io_info(page, wbc, compr_blocks)

    fio.need_lock <- compr_blocks ? LOCK_DONE : LOCK_RETRY

```

```

retry:

atomic_add(PAGE_SIZE, fifs.write_bytes_pending)

err <- call f2fs_do_write_data_page(fio)

if err == -EAGAIN and (sync writeback or from_compress):

call f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT) // 让出 cpu, 准备重试

fio.need_lock <- LOCK_REQ

goto retry

if err != 0:

atomic_sub(PAGE_SIZE, fifs.write_bytes_pending)

break

submitted++

submitted_pages_count <- submitted

return err

```

单动态大页写回函数接收字节粒度的起止范围，循环边界由调用方传入的子范围决定。上层缓存动态大页写回入口先根据逐块脏状态位图查找动态大页内实际变脏的区间，然后只把这些子范围传递给单动态大页写回函数，而不是默认处理整个动态大页。

脏区判定不再依赖动态大页级脏标志，只需要查询保留在逐块脏状态位图中的子块级脏信息，就能找到具体变脏的子区间。随后，针对单大页区间的写回函数利用子区间信息计算动态大页内需要写回的起始索引，循环将每个子页写回。这样，对于一个只有 4KB 子块变脏的 64KB 动态大页，写回范围只会覆盖该 4KB 区间。最终循环体只执行一次，写入量也保持为 4KB，写放大从 16 倍降至 1 倍。

每个子块提交前都会增加未完成写字节计数，块设备 I/O 完成后再按照本次请求长度扣减。只有当该计数归零时，动态大页的写回生命周期才会结束。这一机制保证所有已提交子块的块设备 I/O 都完成之后，写回状态才被解除。

F2FS 数据页写入过程还有一个独特的机制，检查点会尝试获取一把全局写锁，当该锁被获取成功时，数据页写回的过程会获取读锁失败，被迫让出 cpu 并重新尝试对本页缓存写回。然而，原生写回路径会从头提交整个范围，

导致成功提交写回的子块被重复写入。本文把重试范围收敛到失败子块之后：已经成功提交的子块不再重复写入，后续子块从失败位置继续推进。对于同步写回模式和压缩文件写回，重试循环在等待超时后直接回到当前子块的提交点，而不是回退到动态大页起始位置，从而避免前半段已经提交成功、后半段因 checkpoint 重试时仍要整体重写的情况。

普通写回路径与本文子范围控制路径在提交范围、完成判断和重试粒度上的差异汇总于表 4-3，该表已能充分说明动态大页的写放大问题在本文实现的写回路径上得以完全消除。

表 4-3 原生写回路径与子范围控制路径对比

指标	原生写回路径	本文实现写回路径
实际写入量	64KB（整动态大页）	4KB（仅脏子块）
写放大	16x	1x
日志消耗	追加 64KB	追加 4KB
写回完成判定	单次块设备 I/O 请求钩子	未完成写回字节计数 归零
重试粒度	整动态大页	子块级别

4.4.2 压缩文件写回与延迟解锁实现

压缩文件写回复用单动态大页写回函数的核心循环，但需要额外处理簇边界和延迟解锁两个约束。

压缩文件写回首先需要保持簇边界。查找脏范围时，如果当前文件为压缩文件，写回区间会被截断到当前压缩簇末尾。这样单动态大页写回函数每次接收的起止范围都不会跨越簇边界，跨多个簇的动态大页会被拆分为按簇推进的多次写钩子用。每次调用内部仍按子范围控制，只提交实际变脏的子块，从而保持压缩簇的原子语义。

与普通写回不同，压缩写回在扫描到脏子范围后不会立即形成最终块设备 I/O 请求，而是先将其暂存到压缩上下文。此时动态大页不能由单个子页写回函数或单个簇处理过程提前解锁，必须等该动态大页中进入压缩上下文的脏子范围全部被消费后再释放。解锁时机由未完成脏字节计数控制：压缩路径提交整个簇的所有子块后扣减计数，只有计数归零时才解锁动态大页。

主循环结束后还需要处理最后一个仍被锁定的动态大页。若该动态大页没有提交任何子块，例如所有子块都已经是干净状态，说明它只经过了扫描流程

而没有进入实际写回，此时需要手动结束写回状态并释放锁，避免对象长期停留在锁定状态。

4.5 GC 路径动态大页适配实现

GC 搬移路径需要在目标页被标记为脏时准确落到对应子块范围上，否则高阶动态大页会把一次 4KB 搬移放大成整页写回。本节围绕子范围脏区标记、私有标志写入和子页偏移修正说明其适配方式。

下面给出伪代码 4-9，用于说明 GC 搬移在不同阶数动态大页下如何选择整页或子范围脏标记。

伪代码 4-9 move_data_block: GC subrange migration

```

Input: inode, bidx, gc_type

Output: err

folio <- call f2fs_get_lock_data_folio(inode, bidx, true)

if folio is writeback and gc_type == BG_GC:

    return -EAGAIN

call f2fs_set_folio_private_gcging(folio)

if gc_type == BG_GC:

    if folio_order(folio) == 0: // 0 阶 folio 直接设置整体脏标志位

        call folio_mark_dirty(folio)

    else:

        off <- (bidx - folio.index) << PAGE_SHIFT

        call iomap_set_range_dirty(folio, off, PAGE_SIZE)    // 只将子块置脏

        call f2fs_folio_put(folio, true)

    return 0

    call folio_mark_dirty(folio)

err <- call f2fs_do_write_data_page(fio)

if err == -EAGAIN:

    goto retry_fg

    call f2fs_folio_put(folio, true)

return err

```

对于 0 阶动态大页，GC 路径仍沿用原有的整页脏标记；对于高阶动态大页，则根据 GC 选中的目标逻辑块号计算页内偏移，只标记动态大页内对应的 4KB 子范围为脏。这样 4.4 节的单动态大页写回路径读取逐块脏状态位图后，只会提交该 4KB 范围，GC 搬移一个 4KB 子块不再触发整个 64KB 动态大页写回。

同时，GC 迁移标志通过 4.2 节的统一私有标志接口写入扩展型状态对象的柔性数组，而不再直接编码在动态大页私有字段中，从而避免与 iomap 状态指针发生语义冲突。

GC 路径还需要修正块设备 I/O 请求中的子页偏移。读路径向块设备 I/O 请求加入页面时，应根据目标子块在动态大页内部的相对位置计算偏移，使请求只覆盖对应的 4KB 子范围；搬移写路径也需要显式传递起始子页和子页数量，避免继续依赖一个动态大页对应一个文件系统块的旧假设。

4.6 本章小结

本章围绕普通文件区间化读写、扩展型动态大页状态对象、压缩文件读写、子范围写回和 GC 搬移适配，说明了 F2FS 动态大页缓冲 I/O 的关键实现。上述实现共同将普通前台 I/O、压缩文件 I/O、写回和 GC 搬移统一到同一套区间映射、子范围状态和完成计数机制中，避免不同路径分别维护动态大页语义。

第五章 实验设计与性能评估

5.1 测试环境与配置

为验证本文方案在实际运行环境中的效果，实验分别部署在 QEMU 虚拟机与树莓派 5 两款平台上。QEMU 环境用于在受控条件下对比原生 F2FS 与引入动态大页及 iomap 适配后的改造内核在顺序 I/O 场景下的表现；树莓派 5 环境则用于检验同一套改动在真实 ARM 处理器和外接 SSD 上的行为是否与虚拟化环境一致。对应运行状态截图如图 5-1 和图 5-2 所示。

下文两张表分别汇总了两套测试平台的关键配置参数。

表 5-1 QEMU 虚拟机测试环境配置

参数	配置
虚拟化平台	QEMU (aarch64)
内存	4 GB
存储设备	virtio-blk 虚拟磁盘
文件系统	F2FS，挂载选项：defaults,noatime
内核基线	Linux 6.15，原生 F2FS
改造内核	Linux 6.15，引入动态大页+ iomap 适配补丁
测试工具	fio

表 5-2 树莓派 5 测试环境配置

参数	配置
硬件平台	Raspberry Pi 5 (ARM Cortex-A76)
内存	8 GB
存储设备	外接 SATA SSD (通过 USB 3.0)
文件系统	F2FS，挂载选项：defaults,noatime
内核基线	Linux 6.15，原生 F2FS
改造内核	Linux 6.15，引入动态大页+ iomap 适配补丁
测试工具	fio

```
Starting systemd-networkd.service - Network Configuration...
OK ] Started systemd-networkd.service - Network Configuration.
Starting systemd-networkd-wait-onL... Wait for Network to be Configured...
OK ] Found device dev-ttyAMA0.device - /dev/ttyAMA0
OK ] Finished systemd-udev.trigger.service - Coldplug All Udev Devices.
OK ] Found device dev-disk-by\x2duuid-8-bab339-22da-457f-b237-1d65ef85c53c.
Mounting mnt-f2fs.mount - /mnt/f2fs...
Starting ifupdown-pre.service - He...synchronize boot up for ifupdown...
OK ] Finished systemd-networkd-wait-onL... Wait for Network to be Configured.
OK ] Finished ifupdown-pre.service - He...o synchronize boot up for ifupdown.
OK ] Mounted mnt-f2fs.mount - /mnt/f2fs.
OK ] Reached target local-fs.target - Local File Systems.
OK ] Listening on systemd-sysext.socket...tension Image Management (Varlink).
Starting networking.service - Raise network interfaces...
Starting systemd-tmpfiles-setup.se...e Volatile Files and Directories...
OK ] Finished systemd-tmpfiles-setup.se...ate Volatile Files and Directories.
Mounting run-rpc.pipefs.mount - RPC Pipe File System...
Starting rpcbind.service - RPC bind portmap service...
Starting systemd-resolved.service - Network Name Resolution...
Starting systemd-timesyncd.service - Network Time Synchronization...
Starting systemd-update-utmp.servi...ord System Boot/Shutdown in UTMP...
OK ] Started rpcbind.service - RPC bind portmap service.
OK ] Mounted run-rpc.pipefs.mount - RPC Pipe File System.
OK ] Reached target rpc.pipefs.target.
OK ] Reached target rpcbind.target - RPC Port Mapper.
OK ] Reached target nfs-client.target - NFS client services.
OK ] Reached target remote-fs-pre.targe...reparation for Remote File Systems.
OK ] Reached target remote-fs.target - Remote File Systems.
OK ] Finished networking.service - Raise network interfaces.
OK ] Finished systemd-update-utmp.servi...ecord System Boot/Shutdown in UTMP.
Starting modprobe@dm_mod.service - Load Kernel Module dm_mod...
Starting modprobe@efi_pstore.servi... - Load Kernel Module efi_pstore...
Starting modprobe@fuse.service - Load Kernel Module fuse...
Starting modprobe@loop.service - Load Kernel Module loop...
OK ] Started systemd-timesyncd.service - Network Time Synchronization.
OK ] Finished modprobe@dm_mod.service - Load Kernel Module dm_mod.
OK ] Finished modprobe@efi_pstore.service - Load Kernel Module efi_pstore.
OK ] Finished modprobe@fuse.service - Load Kernel Module fuse.
OK ] Finished modprobe@loop.service - Load Kernel Module loop.
OK ] Reached target time-set.target - System Time Set.
OK ] Started systemd-resolved.service - Network Name Resolution.
OK ] Reached target network.target - Network.
OK ] Reached target network-online.target - Network is Online.
OK ] Reached target nss-lookup.target - Host and Network Name Lookups.
OK ] Reached target sysinit.target - System Initialization.
OK ] Started apt-daily.timer - Daily apt download activities.
OK ] Started apt-daily-upgrade.timer - ..y apt upgrade and clean activities.
OK ] Started dpkg-db-backup.timer - Daily dpkg database backup timer.
OK ] Started e2scrub.all.timer - Periodic Metadata Check for All Filesystems.
OK ] Started fstrim.timer - Discard unused filesystem blocks once a week.
OK ] Started logrotate.timer - Daily rotation of log files.
OK ] Started motd-news.timer - Message of the Day.
OK ] Started systemd-tmpfiles-clean.tim...y Cleanup of Temporary Directories.
```

图 5-1 QEMU 运行截图

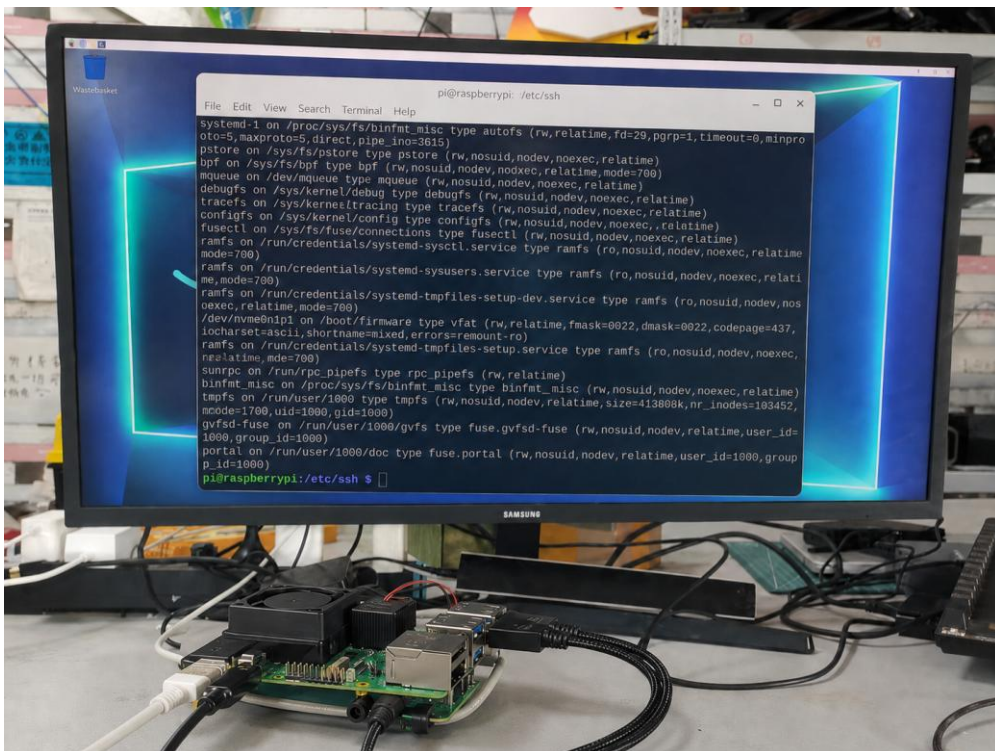


图 5-2 树莓派运行截图

5.2 测试方法与负载设计

5.2.1 工作负载

测试的工作负载使用单线程顺序读写负载，使用系统自带的 fio 测试工具来生成访问请求，分别测试 read 和 write 两种模式。之所以主要采用顺序访问，是因为本文改动的路径集中在页缓存对象粒度、iomap 区间映射以及写回组织方式上，这些变化在连续区间访问中更容易被观察到。相比随机读写，顺序读写可以减少访问位置变化带来的干扰，更适合先判断动态大页路径本身是否起作用。图 5-3 给出了进行一次 fio 调用得到的测试结果输出示例，从输出中可以得到我们想要统计的性能指标。

```
buffered-write: (g=0): rw=write, bs=(R) 1024KiB-1024KiB, (W) 1024KiB-1024KiB, (T) 1024
fio-3.33
Starting 1 process

buffered-write: (groupid=0, jobs=1): err= 0: pid=44340: Thu Aug 14 19:58:44 2025
write: IOPS=5818, BW=5818MiB/s (6101MB/s) (512MiB/88msec); 0 zone resets
--clat (usec): min=108, max=355, avg=155.65, stdev=25.79
--lat (usec): min=119, max=372, avg=169.71, stdev=27.33
--clat percentiles (usec):
--| 1.00th=[ 115], 5.00th=[ 121], 10.00th=[ 126], 20.00th=[ 135],
--| 30.00th=[ 141], 40.00th=[ 147], 50.00th=[ 155], 60.00th=[ 159],
--| 70.00th=[ 165], 80.00th=[ 174], 90.00th=[ 188], 95.00th=[ 202],
--| 99.00th=[ 223], 99.50th=[ 260], 99.90th=[ 355], 99.95th=[ 355],
--| 99.99th=[ 355]
--lat (usec) : 250=99.41%, 500=0.59%
cpu : usr=16.09%, sys=74.71%, ctx=84, majf=0, minf=6
IO depths : 1=100.0%, 2=0.0%, 4=0.0%, 8=0.0%, 16=0.0%, 32=0.0%, >=64=0.0%
submit : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
complete : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
issued rwts: total=0,512,0,0 short=0,0,0,0 dropped=0,0,0,0
latency : target=0, window=0, percentile=100.00%, depth=1

Run status group 0 (all jobs):
WRITE: bw=5818MiB/s (6101MB/s), 5818MiB/s-5818MiB/s (6101MB/s-6101MB/s), io=512MiB (

Disk stats (read/write):
nvme0n1: ios=0/0, merge=0/0, ticks=0/0, in_queue=0, util=0.00%
```

图 5-3 fio 运行结果截图

测试所用文件分为普通文件、稀疏文件和压缩文件三类。普通文件用于观察基本的映射路径，稀疏文件用于检查空洞区间处理是否影响 iomap 路径，压缩文件则对应前文实现中的压缩簇读写适配。另外在冷读测试前会清理页缓存，同时测试文件大小设置为大于可用内存，尽量避免上一轮读取结果留在缓存中影响带宽统计，从而得到更加公平可靠的结果。

5.2.2 测试矩阵

实验围绕以下维度的组合展开：

表 5-3 测试矩阵

维度	取值	说明
文件类型	普通文件、稀疏文件、压缩文件	覆盖普通映射、空洞映射和压缩簇路径
请求块大小	4KB、16KB、64KB、128KB、256KB、512KB、1MB	覆盖从小粒度到大粒度连续访问的完整范围
读写模式	顺序写、冷顺序读、热顺序读	冷读前清理页缓存，热读紧随首轮读取执行

5.2.3 评价指标

实验主要采集以下指标：

表 5-4 评价指标

指标	单位或形式	用途
吞吐量	MiB/s	反映文件系统顺序 I/O 路径的数据传输能力
IOPS	ops/s	辅助衡量小粒度请求下的路径开销
CPU 利用率	用户态与内核态占比	评估改造路径是否引入额外 CPU 开销

5.3 实验结果与分析

5.3.1 QEMU 虚拟机平台

图 5-4 至图 5-6 展示了 QEMU 平台上原生 F2FS 与改造后 F2FS 在普通文件、稀疏文件和压缩文件三类场景下的顺序 I/O 带宽对比。总体来看，改造后路径在普通文件的大块顺序读写场景中提升最明显，在稀疏文件和压缩文件写入场景中也能获得稳定收益。

Bandwidth comparison - normal file - QEMU VM

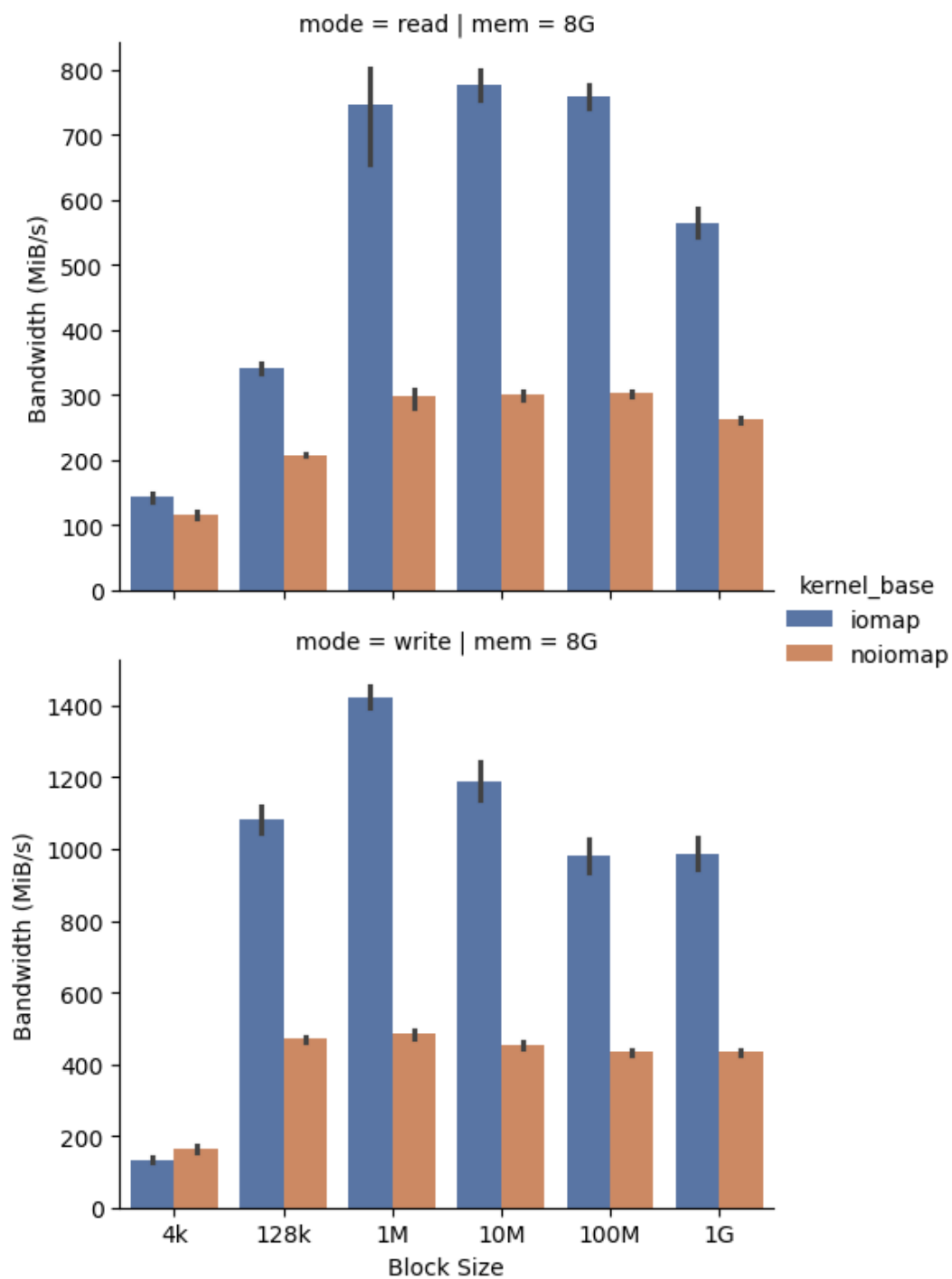


图 5-4 QEMU 平台普通文件顺序 I/O 带宽对比

Bandwidth comparison - sparse file (hole) - QEMU VM

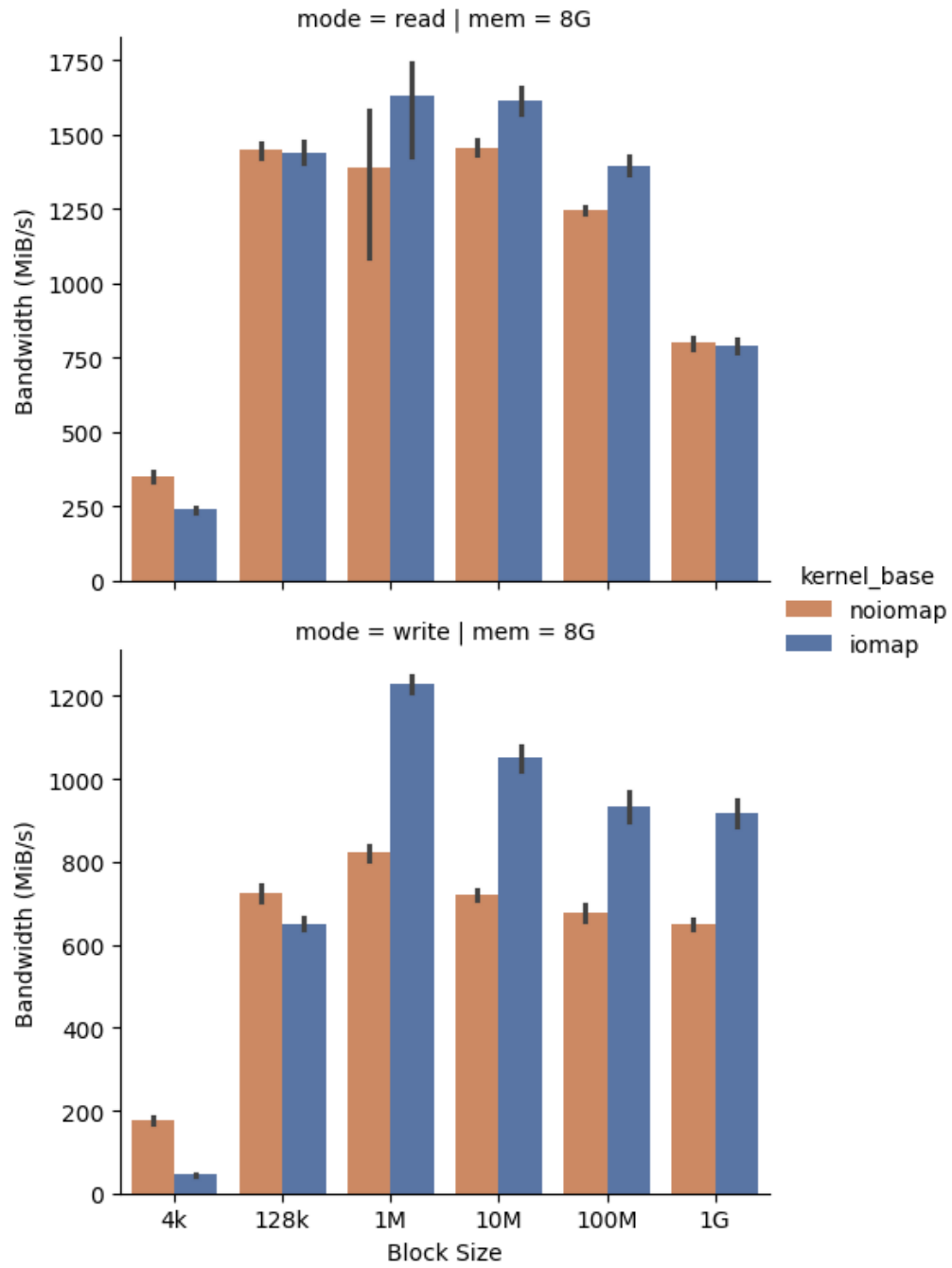


图 5-5 QEMU 平台稀疏文件顺序 I/O 带宽对比

Bandwidth comparison - compressed file - QEMU VM

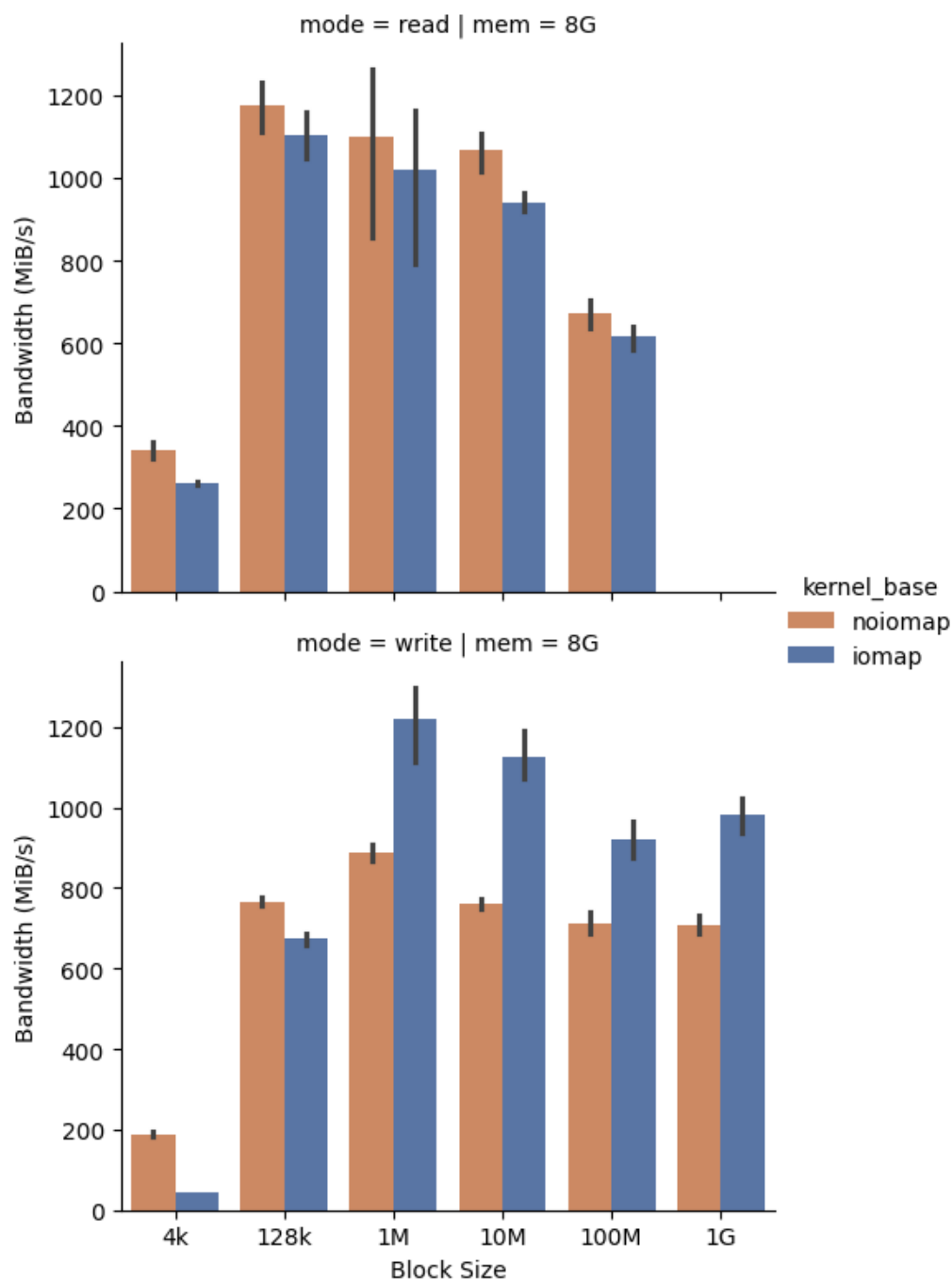


图 5-6 QEMU 平台压缩文件顺序 I/O 带宽对比

普通文件的结果如图 5-4 所示。根据读测试结果，4KB 小请求下两条路径差距不大。然而，随着请求块大小增大，改造前后路径的性能开始拉开差距。128KB 时改造后读带宽约为原生路径的 1.6 倍，1MB 和 10MB 时分别达到约为 2.5 倍和 2.6 倍，1GB 时仍有约 2.2 倍。写测试中类似，4KB 写入时优势不明显，但从 128KB 开始，改造后路径的带宽基本高于原生路径。其中 1MB 请求下提

升最明显，约为原生路径的 2.9 倍；10MB、100MB 和 1GB 下则维持在约 2.3 至 2.6 倍。该结果说明，QEMU 环境下普通文件的连续读写能较直接地受益于更大的页缓存对象和区间化映射。

稀疏文件结果如图 5-5 所示。读测试中，改造后路径的优势没有普通文件明显。1MB 到 100MB 请求范围内，改造后读带宽大约是原生路径的 1.1 至 1.2 倍；4KB 和 1GB 两个点基本持平，个别情况下还略低。该现象可能与空洞区间判断和页缓存填充方式有关，说明稀疏文件读路径中并不是所有开销都能通过动态大页减少。写测试的变化更稳定一些，1MB、10MB、100MB 和 1GB 请求大小下，改造后路径约为原生路径的 1.4 至 1.5 倍。也就是说，稀疏文件在大块写入时仍能利用较大的缓存对象减少部分路径处理次数，但提升幅度低于普通文件。

压缩文件结果如图 5-6 所示。读测试中，改造后路径没有表现出稳定提升，部分块大小下还低于原生路径。从图中可以看到，压缩读的瓶颈不只在页缓存对象数量，还包含压缩簇读取、解压以及解压后数据填充等步骤，因此动态大页对读路径的帮助有限。写测试的结果相对好一些，1MB 请求大小下改造后路径约为原生路径的 1.4 倍，10MB 时约为 1.5 倍，100MB 和 1GB 时约为 1.3 至 1.4 倍。压缩文件写入仍然要遵守簇级提交边界，所以提升幅度不如普通文件，但在大块写入下可以看到一定收益。

5.3.2 树莓派 5 平台

图 5-7 至图 5-9 展示了树莓派 5 平台上相同负载的带宽对比。树莓派 5 的结果与 QEMU 不完全一致，因为该平台通过 USB 3.0 外接 SATA SSD，实际带宽会同时受到 USB 转接、SSD 本身性能和 ARM 处理器执行能力影响。尽管性能指标无法直接跟 QEMU 的对比，但是下面的实验目的主要是比较同一平台上原生路径和改造路径之间的相对变化，以便分析本文所作改造是否在多个平台上面都有效。

Bandwidth comparison - normal file - Raspberry Pi 5

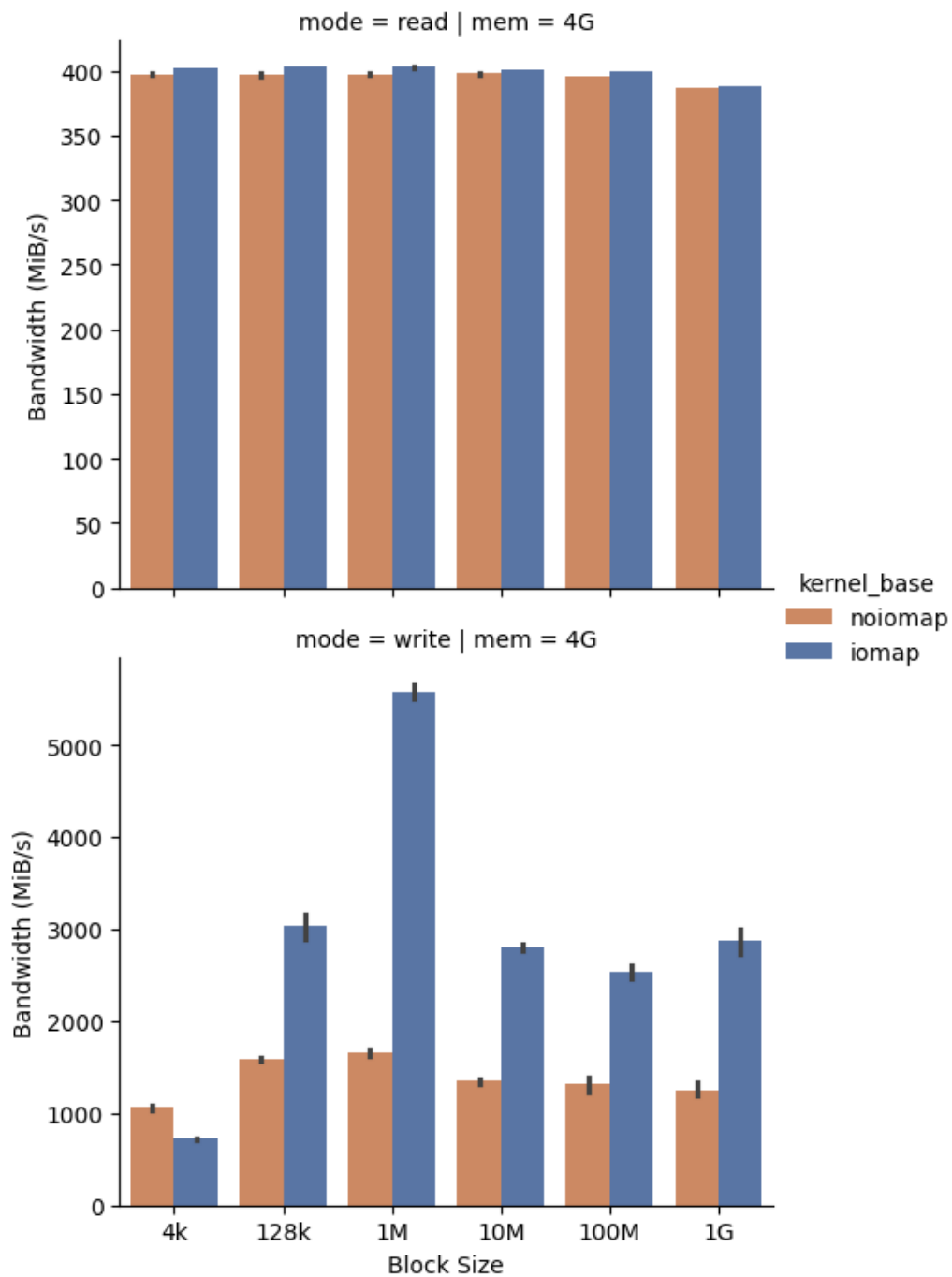


图 5-7 树莓派 5 平台普通文件顺序 I/O 带宽对比

Bandwidth comparison – sparse file (hole) – Raspberry Pi 5

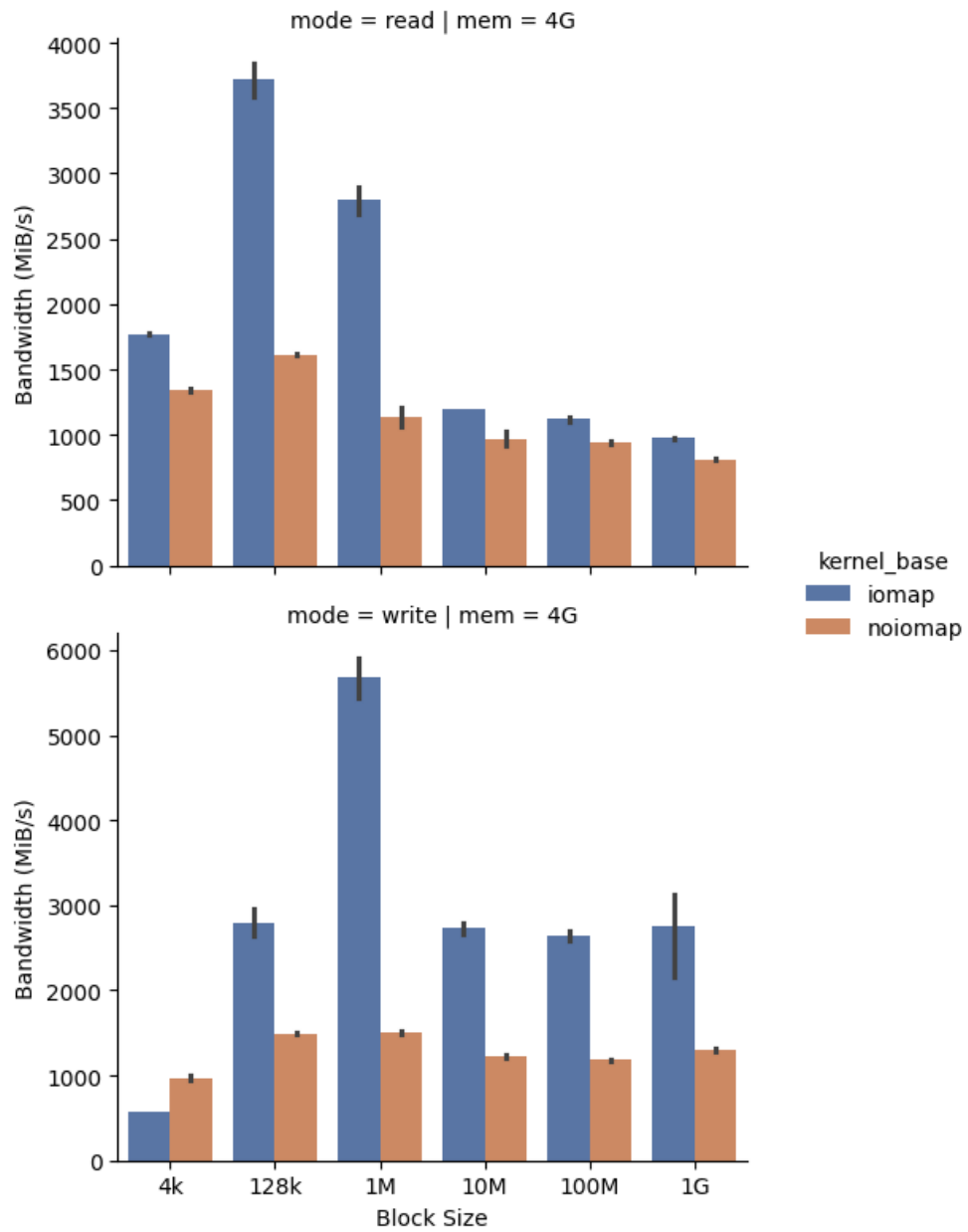


图 5-8 树莓派 5 平台稀疏文件顺序 I/O 带宽对比

Bandwidth comparison – compressed file – Raspberry Pi 5

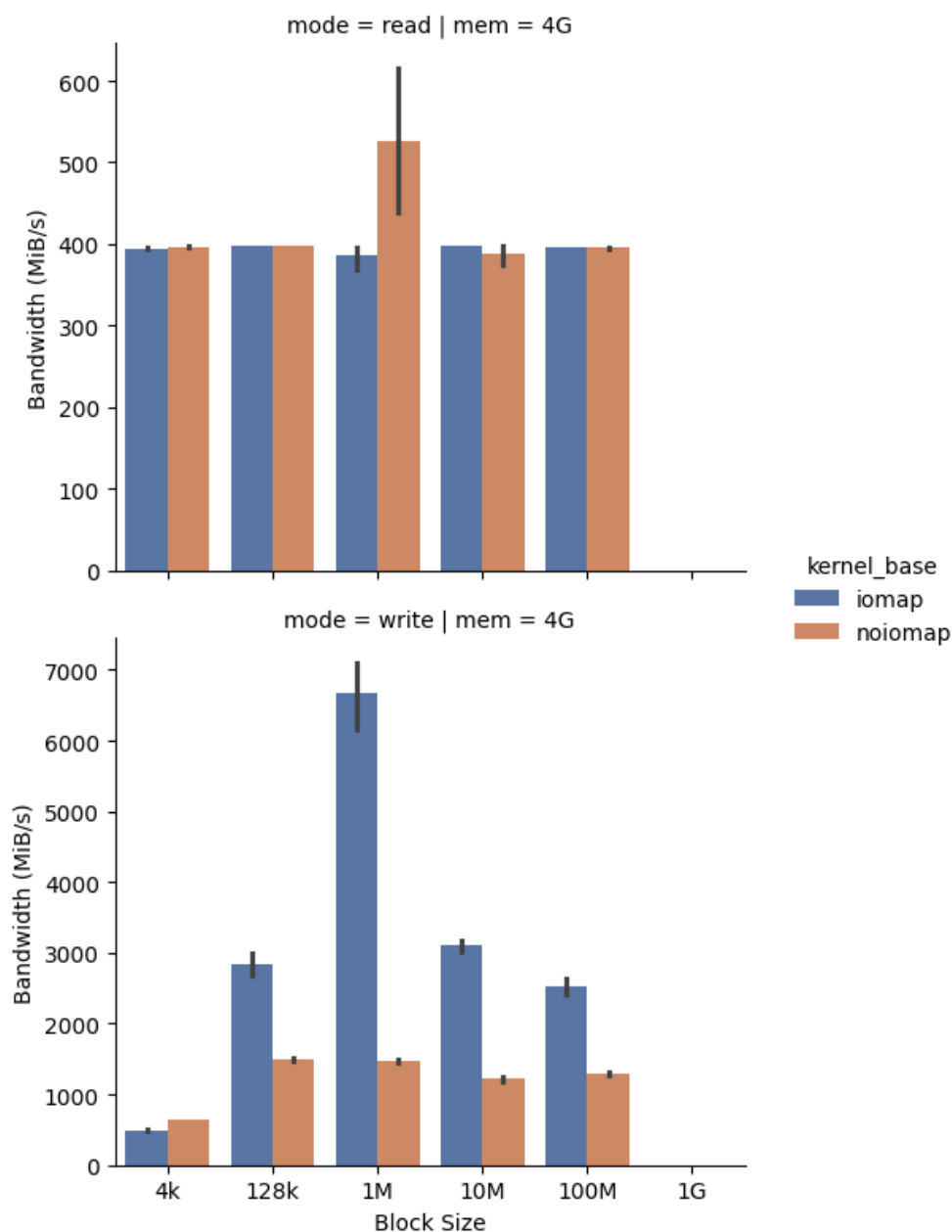


图 5-9 树莓派 5 平台压缩文件顺序 I/O 带宽对比

树莓派 5 普通文件结果如图 5-7 所示。结果显示读测试中，动态大页改造前后的带宽基本接近。推测该平台上的普通文件读可能更多受到 PCIe 2.0 协议的读带宽速度限制。写测试中则差异相当明显，128KB 请求大小下改造后路径约为原生路径的 1.9 倍，1MB 时达到约 3.4 倍，10MB 和 100MB 时分别约为 2.1 倍和 1.9 倍，1GB 时约为 2.3 倍。结合图中柱状变化看，普通文件写入是树莓派平台上收益较明显的一类场景。可以推测，动态大页在缓冲写路径中的正向影响，在真实 ARM 平台上成功转化为吞吐性能的提升。

稀疏文件结果如图 5-8 所示。与 QEMU 中读提升较小不同，树莓派平台上的稀疏文件读出现了比较明显的性能提升。4KB 请求大小下，改造后路径约为原生路径的 1.3 倍；128KB 和 1MB 下分别约为 2.3 倍和 2.5 倍；当请求大小继续增大到 10MB 及以上时，提升幅度回落到约 1.2 倍左右。写测试中，128KB 请求大小下约为原生路径的 1.9 倍，1MB 时约为 3.8 倍，10MB、100MB 和 1GB 时大致保持在 2.1 至 2.3 倍。这里的结果说明，稀疏文件中的空洞处理并没有使动态大页路径失效，但它的收益会随请求块大小变化，并不是单调增加。

压缩文件结果如图 5-9 所示。读测试中，改造后路径整体接近原生路径，多数块大小下约为原生路径的 1.0 倍；1MB 请求大小处反而下降到约 0.7 倍。这个点说明压缩读路径仍然比较敏感，压缩簇大小、解压过程以及页缓存填充方式都会影响最终带宽。写测试中，改造后路径的提升更明显：128KB 请求大小下约为原生路径的 1.9 倍，1MB 时达到约 4.5 倍，10MB 时约为 2.6 倍，100MB 时约为 2.0 倍。由于压缩写入需要按簇组织数据，改造后路径并不是简单扩大写入范围，而是在簇边界内复用动态大页的子范围状态管理。从本组结果看，该处理方式在树莓派平台的大块写入场景中效果比较突出。

图 5-10 至图 5-15 展示了 QEMU 和树莓派两种平台下不同文件类型的带宽稳定性结果。测量带宽稳定性目的是为了衡量动态大页改造带来的性能收益是否相对稳定，不易经受各种随机因素的影响。

Bandwidth stability (box, 10 runs) - normal file - QEMU VM

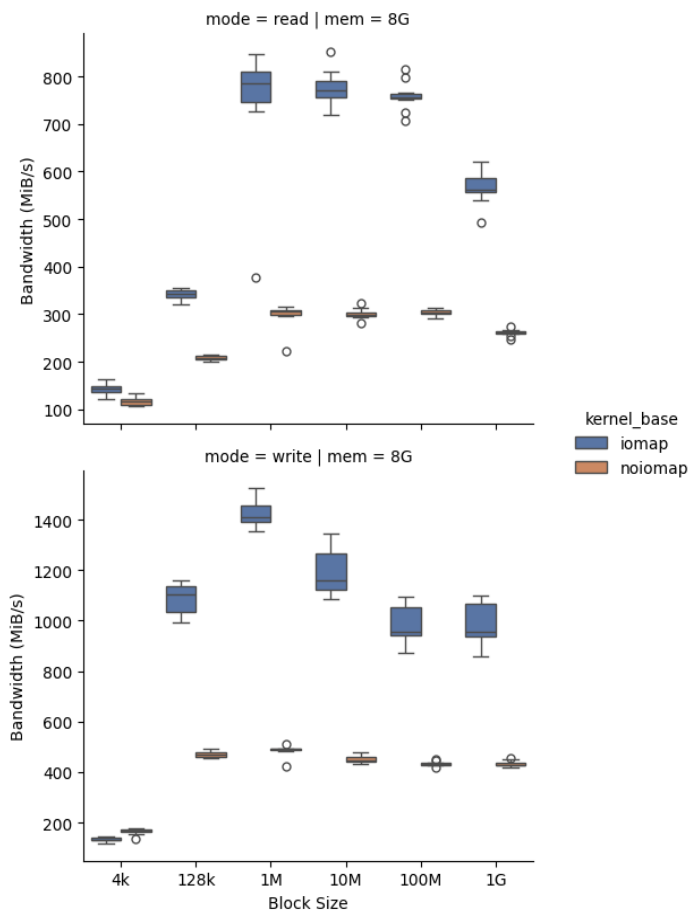


图 5-10 QEMU 平台普通文件带宽稳定性对比

Bandwidth stability (box, 10 runs) - sparse file (hole) - QEMU VM

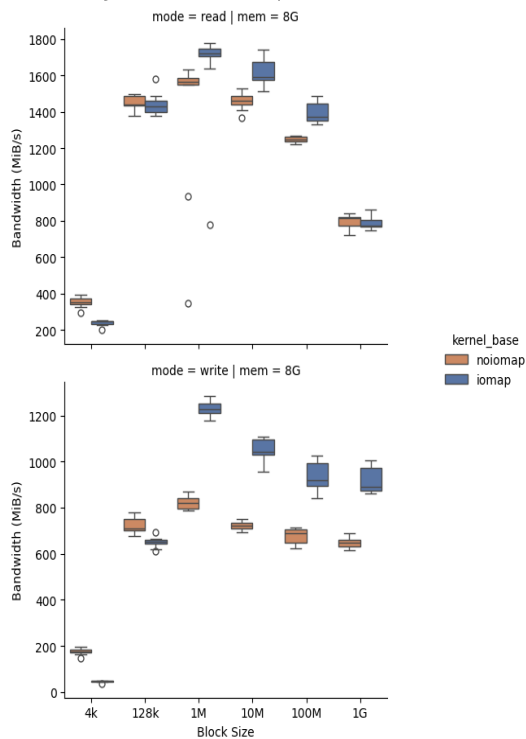


图 5-11 QEMU 平台稀疏文件带宽稳定性对比

Bandwidth stability (box, 10 runs) – compressed file – QEMU VM

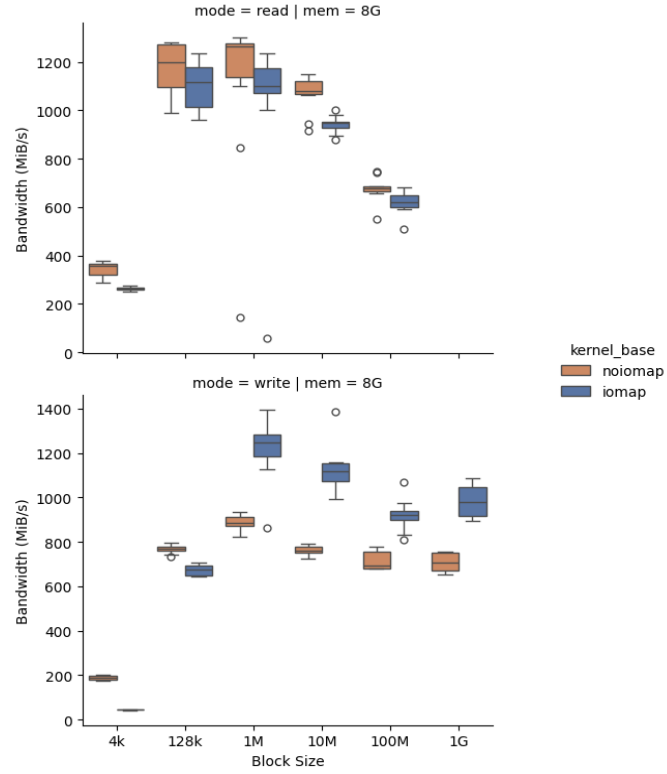


图 5-12 QEMU 平台压缩文件带宽稳定性对比

Bandwidth stability (box, 10 runs) – normal file – Raspberry Pi 5

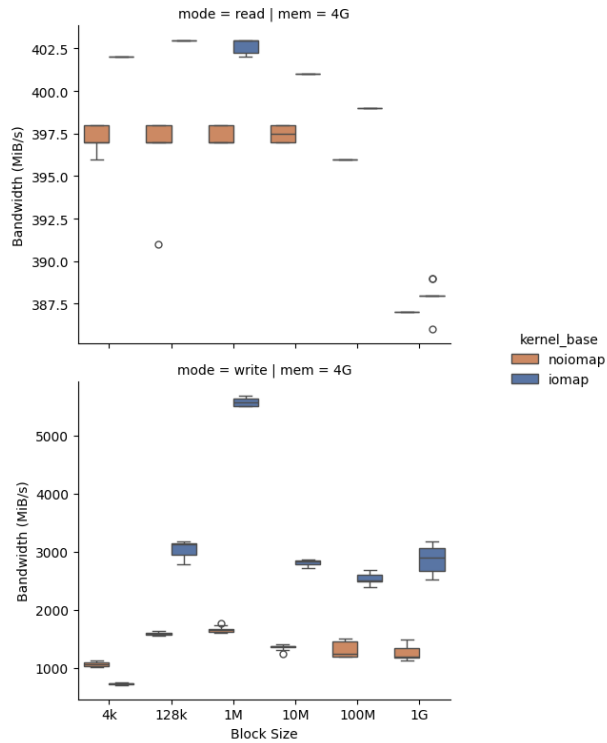


图 5-13 树莓派 5 平台普通文件带宽稳定性对比

Bandwidth stability (box, 10 runs) – sparse file (hole) – Raspberry Pi 5

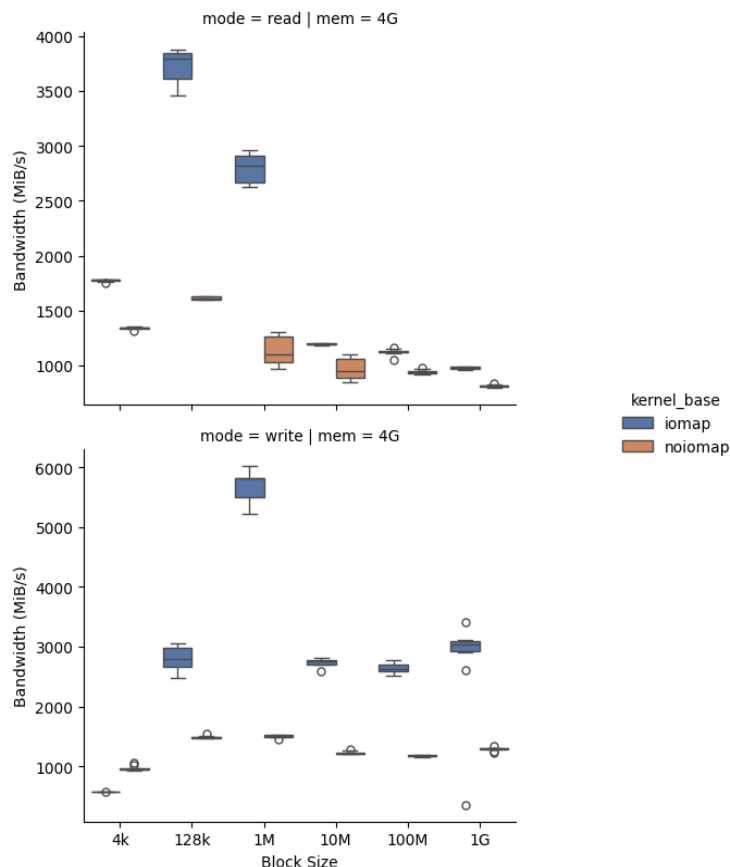


图 5-14 树莓派 5 平台稀疏文件带宽稳定性对比

Bandwidth stability (box, 10 runs) – compressed file – Raspberry Pi 5

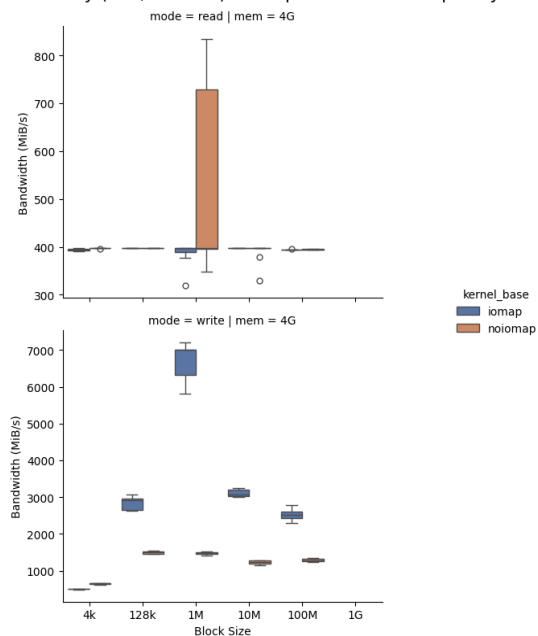


图 5-15 树莓派 5 平台压缩文件带宽稳定性对比

整体来看，改造后路径在大块顺序写场景中不仅平均带宽更高，波动范围也没有明显扩大，说明动态大页和 iomap 路径没有引入额外的不稳定因素。

QEMU 平台中，普通文件的稳定性改善较明显，稀疏文件和压缩文件的波动相对更接近原生路径；树莓派 5 平台中，由于外接 SSD 和 USB 3.0 转接链路会带来一定设备侧波动，部分数据点起伏比 QEMU 更明显，但改造后路径整体仍保持较高带宽水平。压缩文件读场景的稳定性提升不明显，主要原因可能是压缩簇读取、解压和缓存填充过程对带宽影响较大。

5.3.3 CPU 利用率分析

除带宽外，实验还采集了顺序读写过程中的 CPU 利用率，如图 5-16 至图 5-21 所示。整体来看，改造后路径没有因为引入动态大页和 `iomap` 而带来额外的 CPU 占用放大。在 QEMU 普通文件大块负载中，原生路径的内核态 CPU 占比在多个块大小下接近或超过 90%，而改造后路径大多位于约 75% 至 80% 区间；在树莓派普通文件大块负载中，原生路径的内核态 CPU 占比约为 60%，改造后路径多处降至约 50% 左右。该趋势与本文的设计目标一致：动态大页减少页缓存对象数量，`iomap` 以区间迭代替代逐块状态遍历，从而降低了部分小粒度路径管理开销。

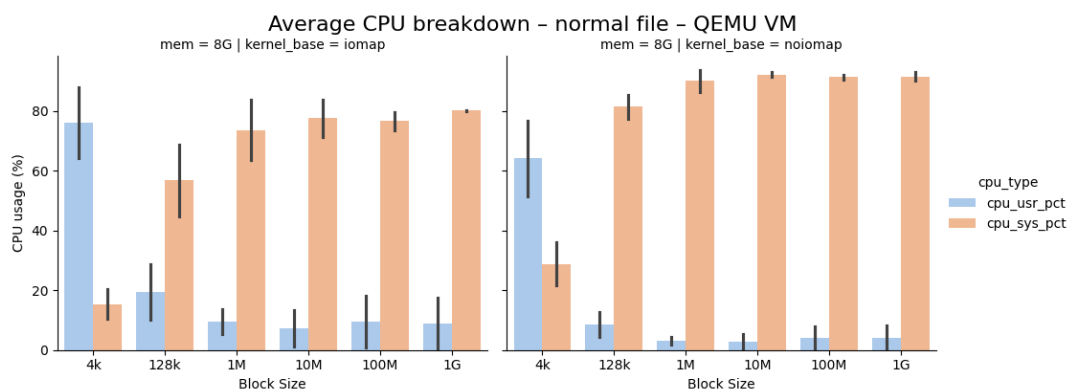


图 5-16 QEMU 平台普通文件 CPU 利用率对比

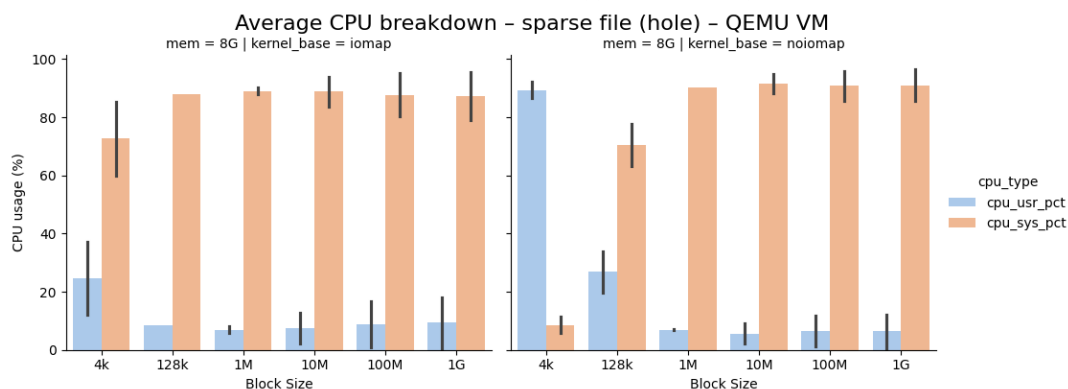


图 5-17 QEMU 平台稀疏文件 CPU 利用率对比

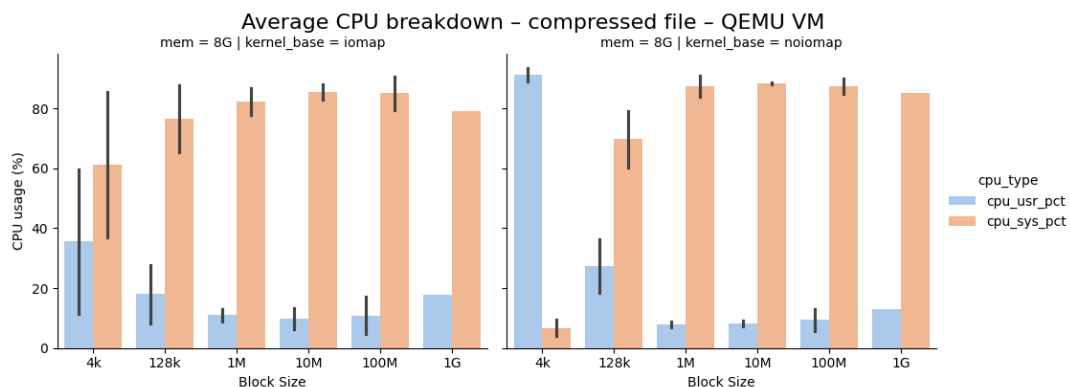


图 5-18 QEMU 平台压缩文件 CPU 利用率对比

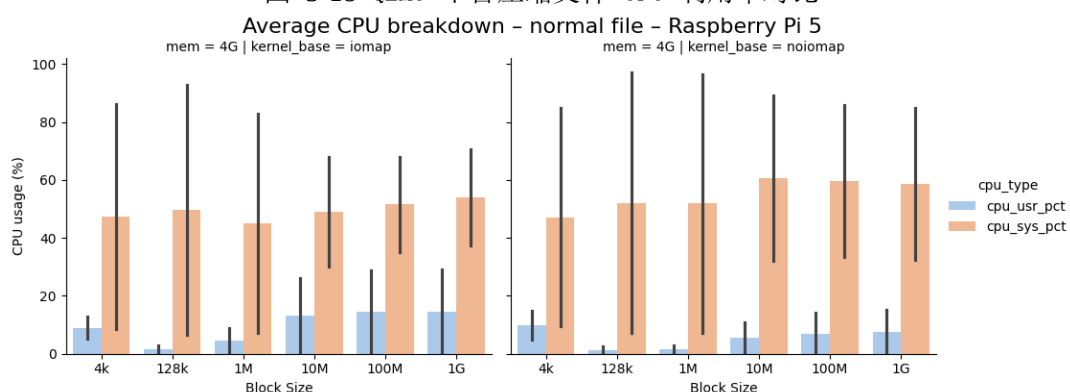


图 5-19 树莓派 5 平台普通文件 CPU 利用率对比

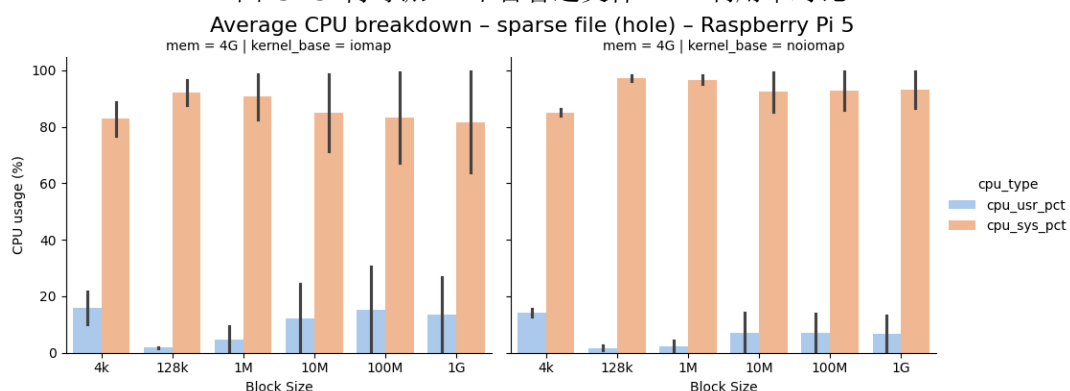


图 5-20 树莓派 5 平台稀疏文件 CPU 利用率对比

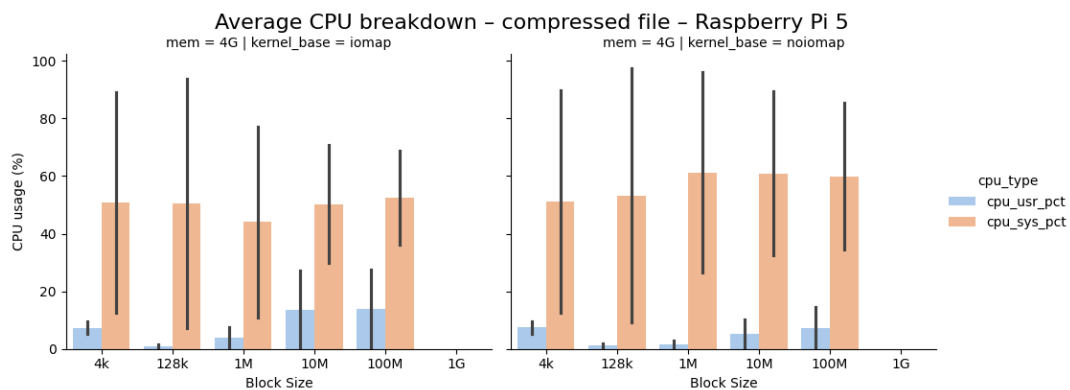


图 5-21 树莓派 5 平台压缩文件 CPU 利用率对比

第六章 总结与展望

6.1 工作总结

本文围绕 F2FS 在动态大页缓存条件下的缓冲 I/O 适配问题展开研究。传统 F2FS 缓冲 I/O 路径长期以 4KB 页缓存对象和 4KB 文件系统块为基本处理单位，普通读写、写回、压缩文件处理和 GC 搬移路径均围绕这一粒度组织状态和完成语义。动态大页机制扩大了页缓存对象的覆盖范围，使单个页缓存对象可能同时包含多个文件系统块，因此需要重新处理块映射、子范围状态、异步完成和专属路径适配等问题。

针对上述问题，本文引入 `iomap` 区间映射模型作为 F2FS 动态大页缓冲 I/O 的组织基础。通过将 F2FS 块映射结果转换为字节粒度的连续区间，普通读写路径能够摆脱逐 4KB 页反复查询和提交的组织方式，并在同一套迭代框架中处理空洞、延迟分配、连续映射和写入预留等状态。同时，本文扩展动态大页状态对象，在页缓存对象内部维护子范围就绪位图、脏状态位图、F2FS 私有标志和未完成 I/O 计数，使一个动态大页内部的多个文件系统块能够被分别跟踪和统一收尾。

在普通文件路径之外，本文进一步适配了 F2FS 中容易与动态大页语义发生冲突的压缩文件路径、子范围写回路径和 GC 搬移路径。在压缩文件路径中，本文主要处理的是压缩簇和动态大页范围不完全一致的问题，在实现中保留了压缩簇边界判断，同时把读写完成情况记录到扩展型动态大页状态对象中，使同一个动态大页内部的不同子范围可以分别完成、统一收尾。写回路径的改动主要集中在 `dirty` 子范围扫描上，对于只修改了部分 4KB 块的动态大页，本文没有直接按整个动态大页提交写回，而是根据 `dirty` 位图确定实际需要落盘的子块范围，再构造对应的 I/O 请求。对于 GC 路径，本文实现了子页偏移计算和搬移范围限制，使后台搬移有效块时仍然保持单块搬移语义，不会因为页缓存对象变大而写错动态大页内部位置。

实验部分使用 QEMU 和树莓派 5 两个平台分别进行了测试，测试对象包括普通文件、稀疏文件和压缩文件，并额外统计了 CPU 利用率。从结果看，本文实现对写路径的帮助更明显，尤其是在普通文件和部分大块顺序写场景中，带宽提升比较稳定。读路径的结果相对复杂一些，普通文件在 QEMU 上有一定提

升，但在树莓派平台上提升不明显；压缩文件读还会受到压缩簇读取、解压和缓存填充过程影响，个别块大小下甚至低于原生路径。这些结果说明，当前实现已经能够在不修改 F2FS 磁盘布局的情况下跑通动态大页缓冲 I/O，并且在连续写入场景中减少了一部分页缓存对象管理和块映射处理开销。但它的收益不是固定的，和请求大小、文件类型、压缩比例、设备带宽以及平台 CPU 能力都有关系。

6.2 未来展望

本文主要围绕固定阶数或典型阶数下的动态大页适配展开，尚未深入讨论不同访问模式下动态大页阶数的自适应选择问题。后续工作可以结合顺序访问、随机访问、压缩文件比例和内存碎片情况，研究面向 F2FS 的动态大页阶数选择策略，使页缓存对象粒度能够根据实际负载变化进行调整。

F2FS 压缩文件路径同时受到压缩簇大小、动态大页大小和块设备 I/O 粒度影响。本文主要解决其在动态大页条件下的状态一致性和完成语义问题，后续仍可以进一步研究压缩簇与动态大页边界不一致时的读写聚合策略，减少跨簇处理带来的额外状态维护和 I/O 分裂。

GC 搬移路径是 F2FS 区别于传统文件系统的重要后台路径。本文已经对 GC 读写中的子页偏移和搬移范围进行了适配，但后台 GC、前台写入和动态大页写回之间仍可能存在调度竞争。后续可以进一步研究动态大页条件下的 GC 触发、搬移批量和写回节奏控制，使后台整理过程与前台 I/O 更好协同。

本文实验已经覆盖 QEMU 虚拟机和树莓派 5 平台，但动态大页对性能的影响与 CPU 架构、内存容量、存储设备和内核版本均有关。后续工作可以在更多硬件平台、更多 F2FS 配置和更新内核版本中进行验证，并进一步评估该方案在长期运行、内存压力和混合负载下的稳定性。

通过上述方向的进一步研究，F2FS 动态大页缓冲 I/O 适配可以从功能可行性验证继续走向更完整的性能优化和工程化实现。

参考文献

[1] Li Z, Zhang G. StreamCache: Revisiting Page Cache for File Scanning on Fast Storage Devices [A]. Bagchi S, Zhang Y. Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC '24) [C]. Santa Clara: USENIX Association, 2024. 1119~1134.

- [2] Kim H, Agrawal N, Ungureanu C. Revisiting storage for smartphones [J]. ACM Transactions on Storage, 2012, 8(4): 1~25.
- [3] Ganapathy N, Schimmel C. General Purpose Operating System Support for Multiple Page Sizes [A]. Proceedings of the 1998 USENIX Annual Technical Conference (USENIX ATC '98) [C]. New Orleans: USENIX Association, 1998.
- [4] Winwood S, Shuf Y, Franke H. Multiple Page Size Support in the Linux Kernel [A]. Proceedings of the Ottawa Linux Symposium (OLS '02) [C]. Ottawa: Ottawa Linux Symposium, 2002. 573~593.
- [5] Weisberg P, Wiseman Y. Using 4KB Page Size for Virtual Memory is Obsolete [A]. Proceedings of the 10th IEEE International Conference on Information Reuse and Integration (IEEE IRI '09) [C]. Piscataway: IEEE, 2009. 262~265.
- [6] Google. 16 KB page size [EB/OL]. <https://source.android.com/docs/core/architecture/16kb-page-size/16kb>, 2026-04-20.
- [7] Google. Support 16 KB page sizes [EB/OL]. <https://developer.android.com/guide/practices/page-sizes>, 2026-05-23.
- [8] Navarro J, Iyer S, Druschel P, Cox A. Practical, transparent operating system support for superpages [A]. Draves R, van Renesse R. Proceedings of the 5th Symposium on Operating Systems Design and Implementation (OSDI '02) [C]. Boston: USENIX Association, 2002. 89~104.
- [9] Panwar A, Prasad A, Gopinath K. Making huge pages actually useful [A]. Gupta R, Amarasinghe S. Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '18) [C]. New York: ACM, 2018. 679~692.
- [10] Panwar A, Bansal S, Gopinath K. HawkEye: Efficient Fine-grained OS Support for Huge Pages [A]. Bahar I, Herlihy M. Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19) [C]. New York: ACM, 2019. 347~360.
- [11] The Linux Kernel Organization. VFS iomap [EB/OL]. <https://docs.kernel.org/filesystems/iomap/>, 2026-04-20.
- [12] Mathur A, Cao M, Bhattacharya S, et al. The new ext4 filesystem: current status and future plans [A]. Hutton A. Proceedings of the Linux Symposium (OLS '07) [C]. Ottawa: OLS, 2007. 21~33.

- [13] Rodeh O, Bacik J, Mason C. Btrfs: The Linux B-tree Filesystem [J]. ACM Transactions on Storage, 2013, 9(3): 1~32.
- [14] Joshi K, Gupta A, González J, et al. I/O Passthru: Upstreaming a Flexible and Efficient I/O Path in Linux [A]. Ma X, Won Y. Proceedings of the 22nd USENIX Conference on File and Storage Technologies (FAST '24) [C]. Santa Clara: USENIX Association, 2024. 107~121.
- [15] Lee C, Sim D, Hwang J-Y, Cho S. F2FS: A New File System for Flash Storage [A]. Schindler J, Zadok E. Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST '15) [C]. Santa Clara: USENIX Association, 2015. 273~286.
- [16] Rosenblum M, Ousterhout J K. The design and implementation of a log-structured file system [J]. ACM Transactions on Computer Systems, 1992, 10(1): 26~52.
- [17] The Linux Kernel Organization. Flash-Friendly File System (F2FS)[EB/OL]. <https://docs.kernel.org/filesystems/f2fs.html>, 2026-04-20.
- [18] Zhang J, Shu J, Lu Y. ParaFS: A Log-Structured File System to Exploit the Internal Parallelism of Flash Devices [A]. Gulati A, Weatherspoon H. Proceedings of the 2016 USENIX Annual Technical Conference (USENIX ATC '16) [C]. Denver: USENIX Association, 2016. 87~100.
- [19] KernelNewbies. Large block sizes (LBS)[EB/OL]. <https://kernelnewbies.org/KernelProjects/large-block-size>, 2026-05-23.
- [20] Raghav P. Filesystem support for block sizes larger than the page size[EB/OL]. <https://lwn.net/Articles/1009548/>, 2026-05-23.
- [21] Bovet D P, Cesati M. Understanding the Linux Kernel [M]. 3rd Edition. Sebastopol: O'Reilly Media, 2005.
- [22] Gorman M. Understanding the Linux Virtual Memory Manager [M]. Upper Saddle River: Prentice Hall, 2004.
- [23] Rubini A, Corbet J, Kroah-Hartman G. Linux Device Drivers [M]. 3rd Edition. Sebastopol: O'Reilly Media, 2005.
- [24] Hildenbrand D, Schulz M, Amit N. Every Mapping Counts in Large Amounts: Folio Accounting [A]. Bagchi S, Zhang Y. Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC '24) [C]. Santa Clara: USENIX Association, 2024. 1273~1282.

致谢

感谢我的导师在操作系统方面的悉心教学以及选题方面的指导,让我拥有了扎实的基本功以及明确的努力方向,并且感谢导师的持续鼓励,让我在面对晦涩复杂的 Linux 内核代码能鼓起勇气和信心,坚持到底。

同时感谢我的家人和朋友在这段时间里对我的鼓励和支持。论文完成过程中有很多反复修改和调试的阶段,也有不少因为实验结果不稳定而感到焦虑的时候。家人的理解让我能够安心投入,朋友的交流和陪伴也帮助我在压力较大的时候重新调整状态。感谢他们一直以来的支持,使我能够坚持完成这项工作。